

Data = raw facts

Database = Collection of raw facts

* It is defined as collection of related data.

* It can either be computerized or manual.

AIM → To know computerized data.

DBMS

* It is a software which controls or enables the user to create and maintain a database.

* It is a general purpose software system, that facilitates the method of defining, constructing, manipulating and sharing among various users & applications.

Defining → To provide a particular representation either in tree, queue or graph.

Constructing → To arrange or put data in particular order.

Manipulating → To delete, edit, insert the stored data.

Sharing → To share ^{the} stored data between ~~the~~ different systems.

Types of DBMS

* According to user it is divided into :-

(i) Single User

(ii) Desktop Database

(iii) Multi User

(iv) So User

(v) Enterprise

* According to location:-

- (i) Centralised DBS → has single location
- (ii) Distributive DBS → has more than one location

* According to type of data stored:-

- (i) General purpose DBS
- (ii) Discipline Specific

* According to type of use:-

- (i) Operational DB → • concentration on the day-to-day data of company

- There is OLTP (Online Transactional Processing) for it.

- (ii) Analytical DB → • Storing historical data and analysis

- There are decision making capabilities

- for it we have:- # OLAP
(Online Analytical Processing)
Data Warehouse

* According to Structure:-

- (i) Unstructured → Raw unstructured data is used.
- (ii) Structured → Data is formatted before use
- (iii) Semi-Structured

Characteristics of Database

- * Self-describing in nature.
- * Insulation b/w programs and data and data abstraction.
- * Support of multiple views.
- * Sharing of data and multiuser transaction processing.
- * Data redundancy (Data is minimised)
 ↳ Duplication of Data (is removed)
- * Security

6.2.23

DB users and Administrators

DBA (Database Administrator) (1st User)

- * Authorised access to the DB
- * Coordinating and monitoring DB use
- * Acquiring software and hardware resources as needed.
- * Security
- * If DB has poor response time, DBA is answerable.
- * First user of DB system is DBA.

DB Designer (2nd User)

- * Identifies data to be stored in DB
- * Choosing appropriate structure to store and represent data.

End User

- * Queries DB system
- * Updates DB
- * Generates report

The End-User can be of different types :-

- (i) Casual End-User :- Managers, use the DB when required.
- (ii) Naive and Parametric User :- A sizeable portion of DB is to be used by that user.

The sizeable DB is called canned transaction

- (iii) Sophisticated End-User :- The DB is used by these user when it is highly required.

- (iv) Standalone User :- Maintains personal DB by using readymade programmable DB.

- (v) System Analysts and Application Programmer :- Application programmer tests & debugs the program.

- (vi) Workers behind the scene :- They will not interact with database directly. They are :-

- (i) DBMS System Designer and Implementors

- (ii) Developers / Tool developers

- (iii) Operators and maintenance maintenance Personnel.

Advantages of using DBMS approach :-

- No redundancy unlike the file system, i.e., no duplication of data.
- Restricting unauthorized access.
- Providing persistent storage for prog. objects.
- Providing storage structure for efficient query processing.

- providing back-up and recovery.
- providing multiple user interface.

Database Systems concepts and Architecture

Data Model :- Collection of models used to describe the structure of DB.

Data abstraction :- The way to suppress the actual usage of data.

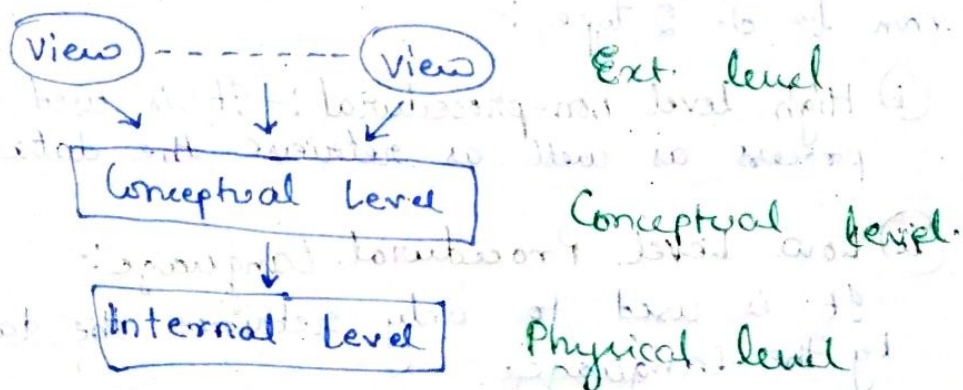
Schema :- Description of DB is called Schema.

- * The displayed schema is called schema diagram.
- * Each object in schema diagram is schema construct.

DB state or snapshot :- Define data in a DB in a particular time is called DB state.

Architecture of Database

It is defined by 3-level. The aim of architecture is to separate program area from physical storage.



3-level architecture of DBS

Data Independence :- Capacity to change schema at one level without affecting the schema at other level.

Data Independence is of two types :-

- (i) **Logical :-** Doing changes in conceptual level without changing the viewing level.

(ii) Physical :- Changes in internal schema without affecting the conceptual level. It is earlier than logical.

Disadvantages of 3-level architecture

DB Languages

DDL (Data Definition Language is needed)

* This language is used to create DB

* It is used at conceptual level.

SDL (Storage Definition Language)

* This language is used to operate with internal schema.

VDL (View Definition Language)

* To manipulate the view level of data.

DML (Data Manipulation Language)

* It is used to manipulate the data

* It can be of 2 type :-

(i) High Level non-procedural :- It is used to process as well as retrieve the data.

(ii) Low Level Procedural Language :-
It is used to only retrieve the data by the language.

Interface of DBMS

9.2.23

① Menu based Interface

The user has a list of menu.

② Form based Interface

Each user is provided with the form. There is form specific language.

⑤ Graphical User Interface

All the information is stored in the form of image.

⑥ Natural Language Interface

The language like English is used for the information storage.

⑦ Speech Input & Output

The telephonic input and output is used for information storage and propagation.

⑧ Parametric user Interface

Works with sizeable amount of data each day.

⑨ Interface for DBA

The interface by which DBA interacts with system.

⑩ DBMS Component Module

Level 1 :- User

- DBA staff
- Casual User
- Application Programmer
- Parametric Users

DDL Statement

DDL Compiler

Privileged Commands

System Catalog
or
Data Dictionary

Casual User

Interactive Query

Query Compiler

Query Optimiser

Application Programmer

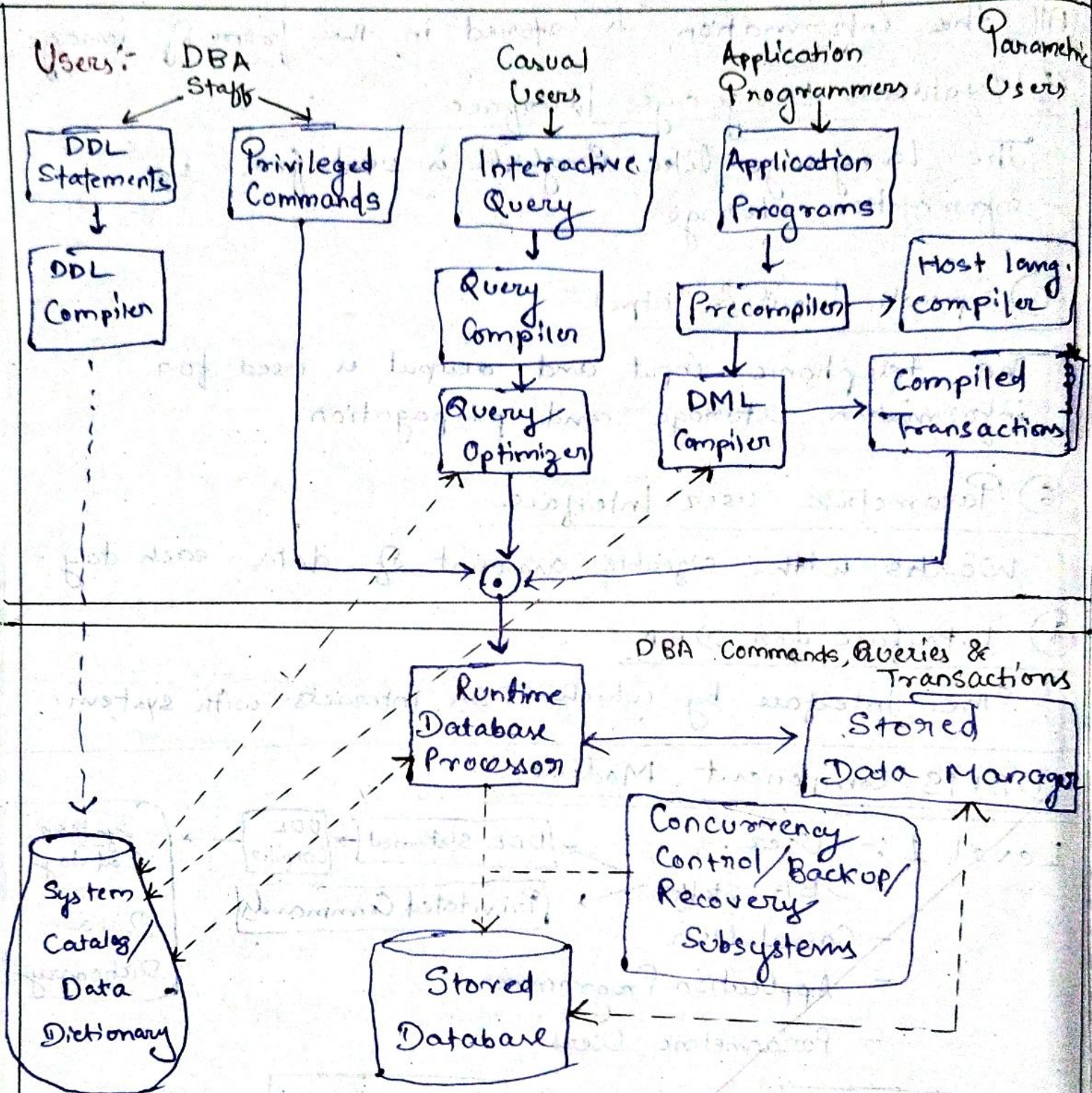
Pre Compiler

Host lang. Compiler

DML Compiler

Compiled transaction

Database Component Modules



Database System Utilities

~~(i) Loading~~ Common utilities have following types of functions :-

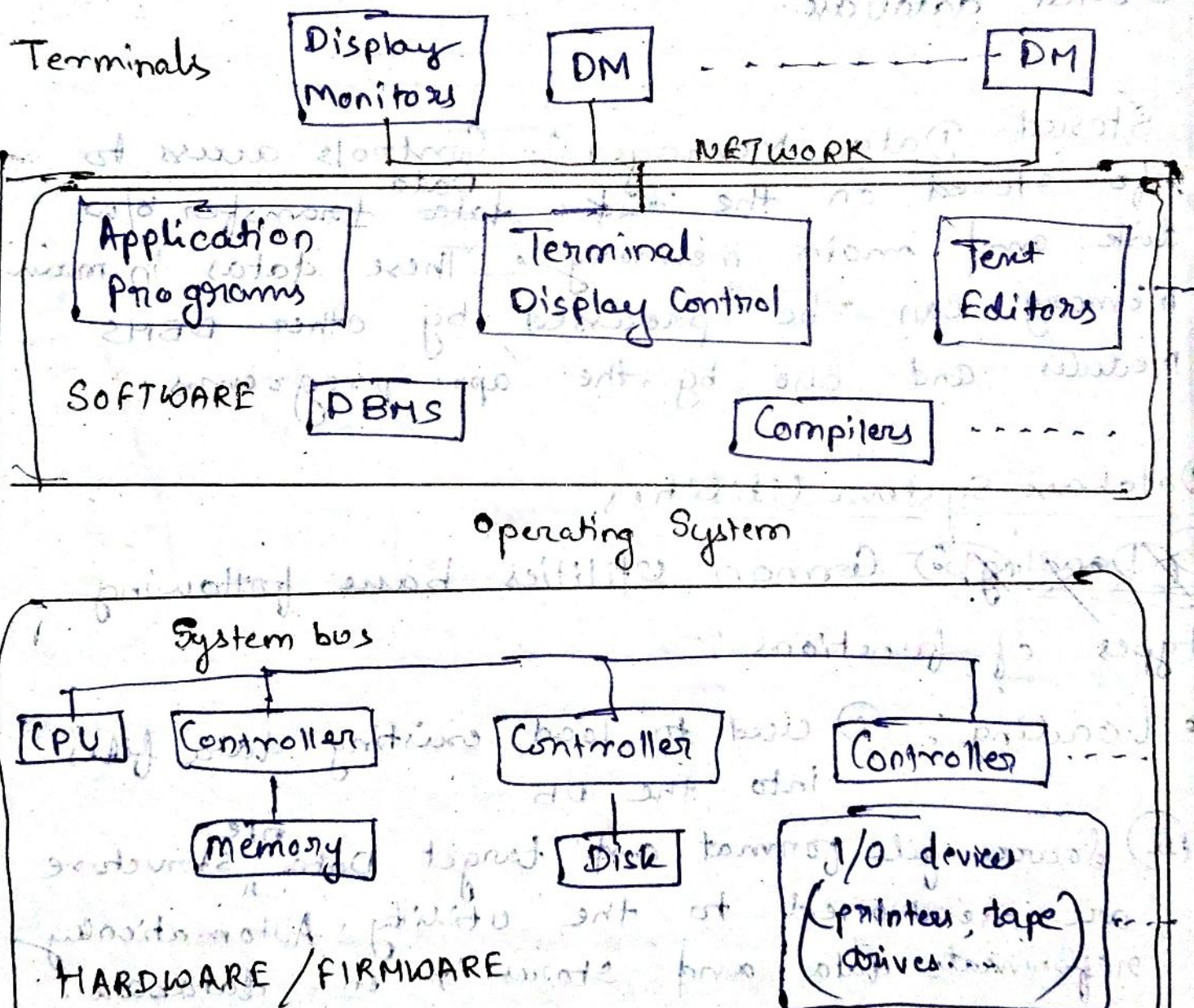
- Loading :- (i) Used to load existing data files into the DB
- (ii) Source file format and target data ^{file} structure are mentioned to the utility. Automatically reformats data and stores it in database.
- Backup :- (i) Creates a backup copy of DB (from any kind of sys. failure)
- (ii) Incremental backups are often used. Though it is complex, it saves space.
- * Incremental backup → Previous backups are recorded too

- File reorganization:
 - (i) Used to reorganize a DB file into different file organization. Improves performance.
- Performance monitoring:-
 - (i) Monitors DB usage and provides statistics to DBA.

Architecture of DBMS :-

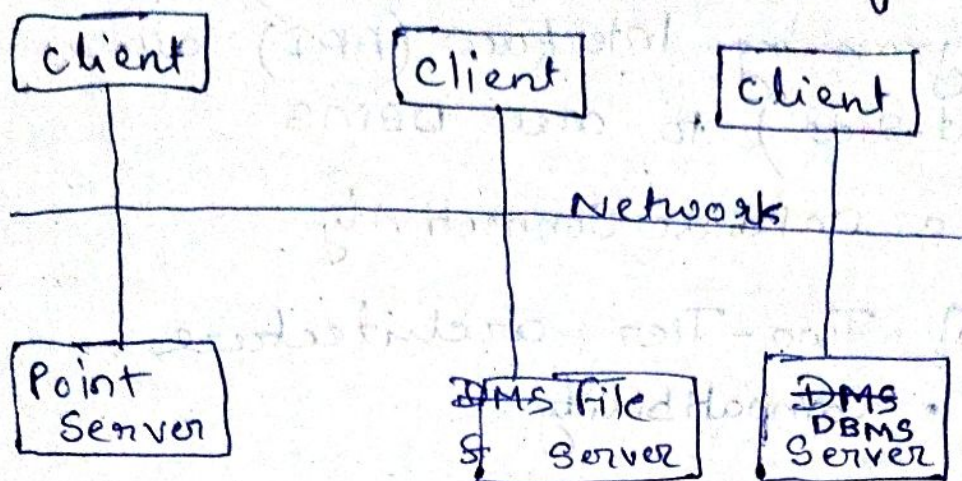
(1) Centralised Architecture

All modules in centralised manner



② Basic client/server Architecture

We have different client for this architecture



* Client
↓
user machine
providing user
Interface capabilities &
local processing

(ii) Three-Tier Client Server Architecture

Additional intermediate layer b/w client & DB server.

Client:

GUI,
Web Interface

Stores rules
used to access
data from DB
server.

App. Server
or
Web Server

Application
Programs, Web
Pages

Accepts and processes
requests from
clients

DB
Server

Database
Management
System

Sends DB command
to DB server.

App. Server or Web server also acts as a channel for passing processed data from DB server.

In client side, data can be processed and filtered further to make it more presentable (i.e. Graphical User Interface)

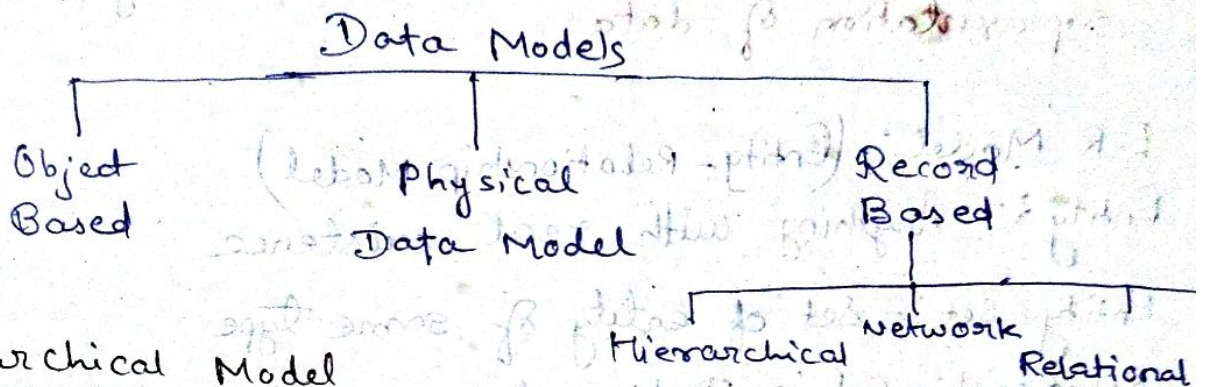
Data Models

There are diff. models of DB :-

- ① Hierarchical ② Networks ③ Relational

Each model has 3 subcomponents :-

- (i) Structural part (ii) manipulative part
(iii) integrity part (data accuracy)



Hierarchical Model

- The data defined by relay structure.
- It is parent child model.

Network Model

- The data is to be stored with links.
- The searching is easier.

Relational Model

- It is a table like structure where we can maintain a record in terms of attribute.
- Redundancy is reduced

Model Selection

It is done on: the basis of

- (i) Organization's primary goal and requirements.
- (ii) Volume of transaction
- (iii) Estimated no. of enquiry

* The relational model is best among all for representation of data.

E-R Model :- (Entity-Relationship Model)
Entity :- Anything with

Entity :- Anything with real existence

Entity Set:- Set of entity of same type

Relationship Set :- Defines the association with entities

Attributes :- The properties of the entity given by attributes.

Different types of attributes in a entity set :-

- 1) Composite v/s simple

- ① Composite v/s simple

A particular attribute which can be sub-divided.
eg → sheet address → city

eg → sheet address → city
state
zip

- ② Stoned v/s Derived

Eg → Birthday → stored, when Birthday is input
Age → derived

- ③ Complex

It is composite as well as multivalued.
Eg → Address - Phone No.

Eg → Address - Phone { Phone (Area-code, Phone no.) } multivalued.

Entering of null values with attribute list :-

There are 3 cases in which it is used :-

- (i) forgot data (ii) No idea about data (iii) Not Valid
- It is usually recommended to avoid using null value.

Relationship Types, Sets :-

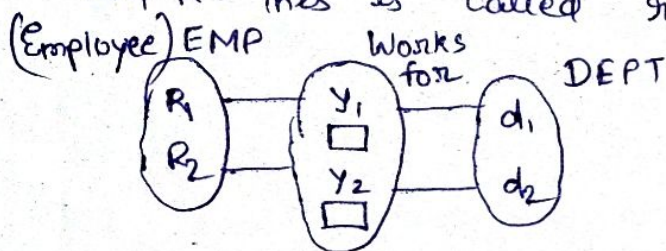
When R is relationship type only when it shows relation between entities.

eg. $\rightarrow R, n$ entity types
 $E_1, E_2, \dots, E_n$

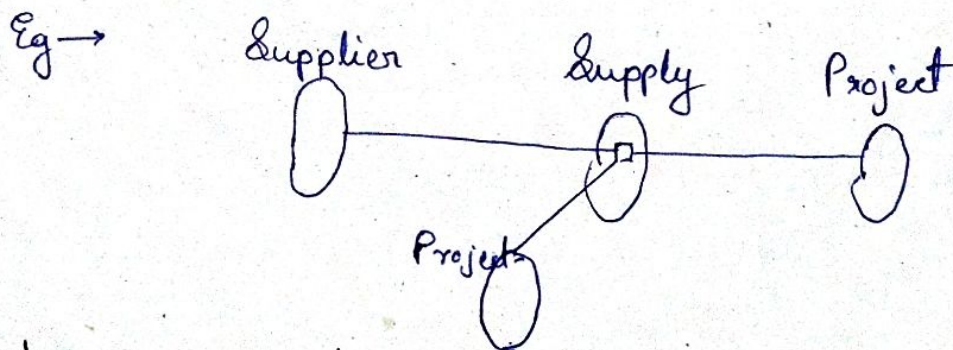
Relationship Instance :-

There is a subset $(e_1, e_2, \dots, e_n)$

$x_i \Rightarrow$ defines the relationship between the subset and R . This is called relationship instance.



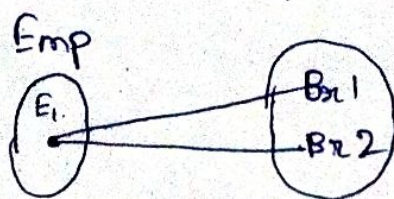
Degree of relationship :- It defines no. of entities participated in a particular relationship.



Role names and recursive relationship

- The same entity to define a relationship is called recursive relationship.

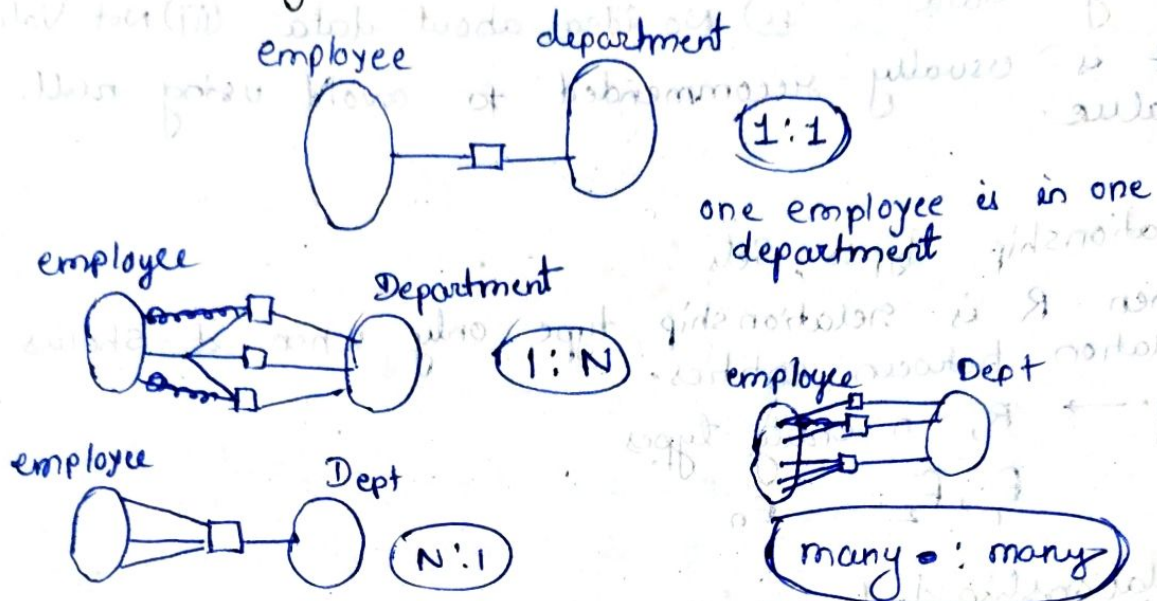
eg. $\rightarrow$ Supervision Supervisions Subordinate



Constants of Relation types :-

13.2.23

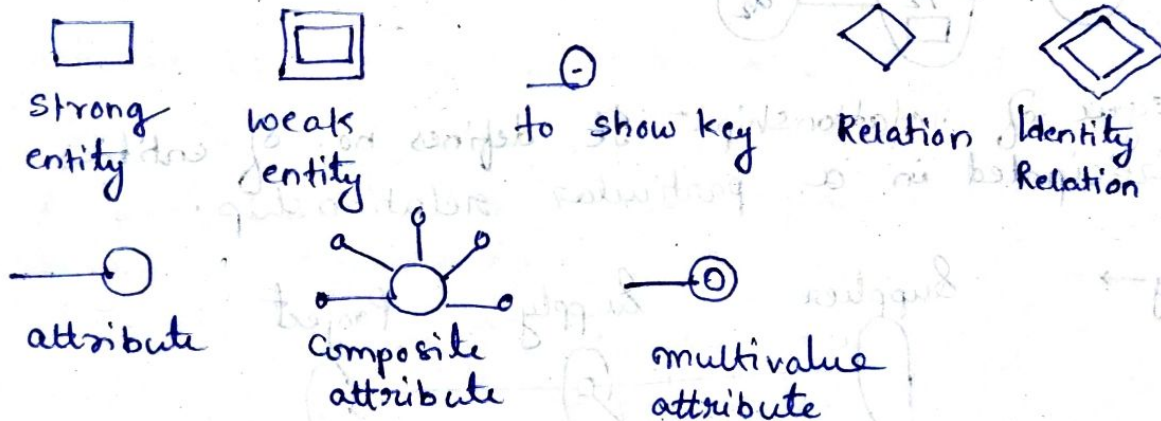
Cardinality Ratio



Participating constraints and existing dependency

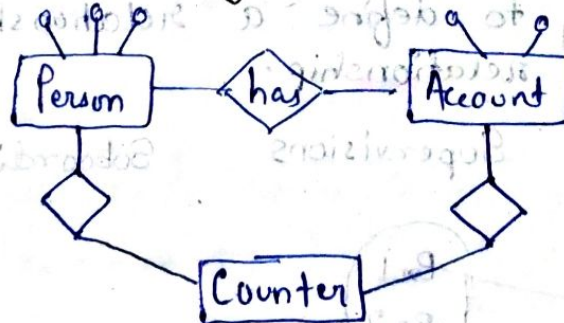
Total Partial

Weak Entity Type

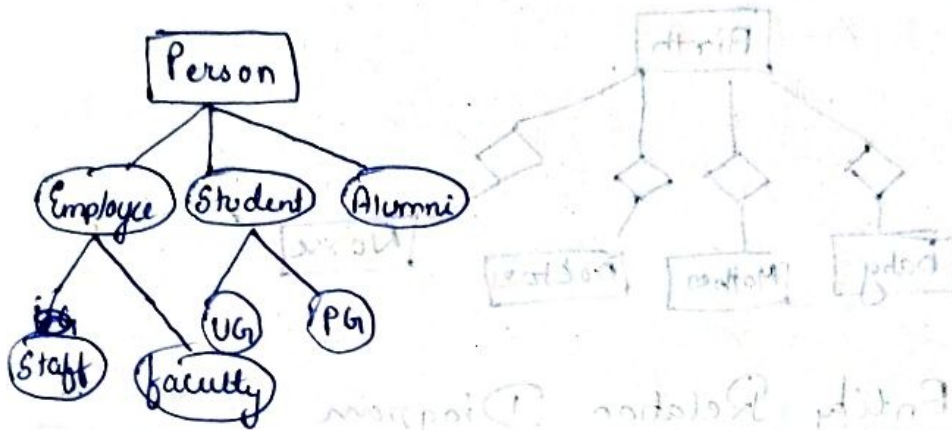


Q. A person has account in bank, bank operates on diff. counter, person gets loan from counter. Draw diagram.

Ans :

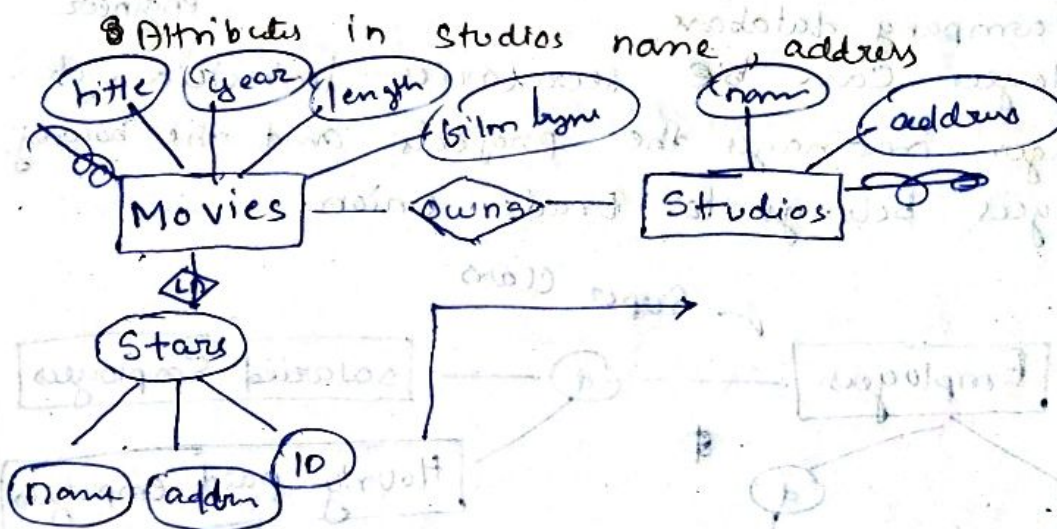


- Q A person is either an employee, alumni or a student. Employee can either be staff or faculty. Student can be either P.G or UG. Show entity diagram.



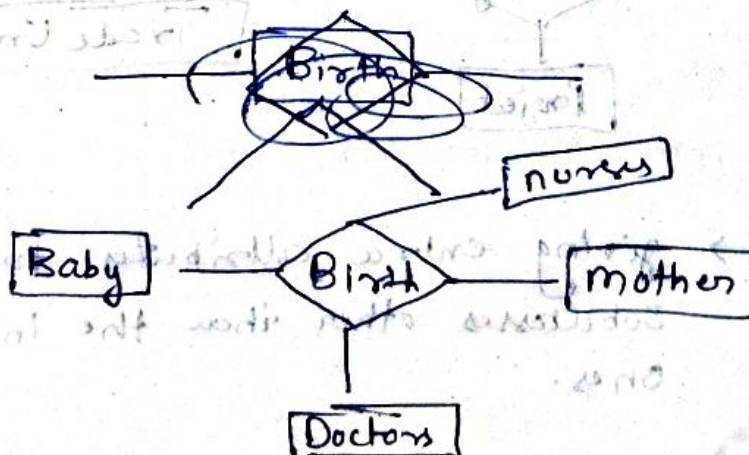
16.2.23

- Q Stars in movies. Movies own studios.
 Attributes in movies: title, year, length, film type
 Attributes in stars: name, address, ID

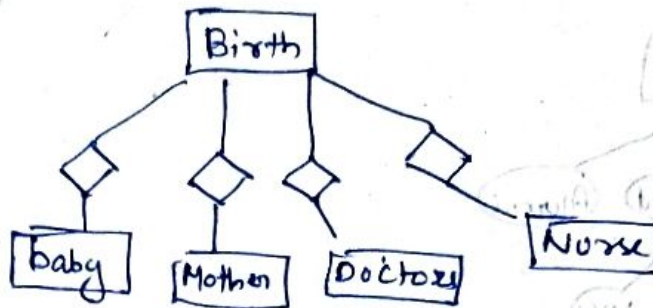


In a birth,

- Q One baby, one mother, no. of nurses and no. of doctors.



Redrawing birth as entity



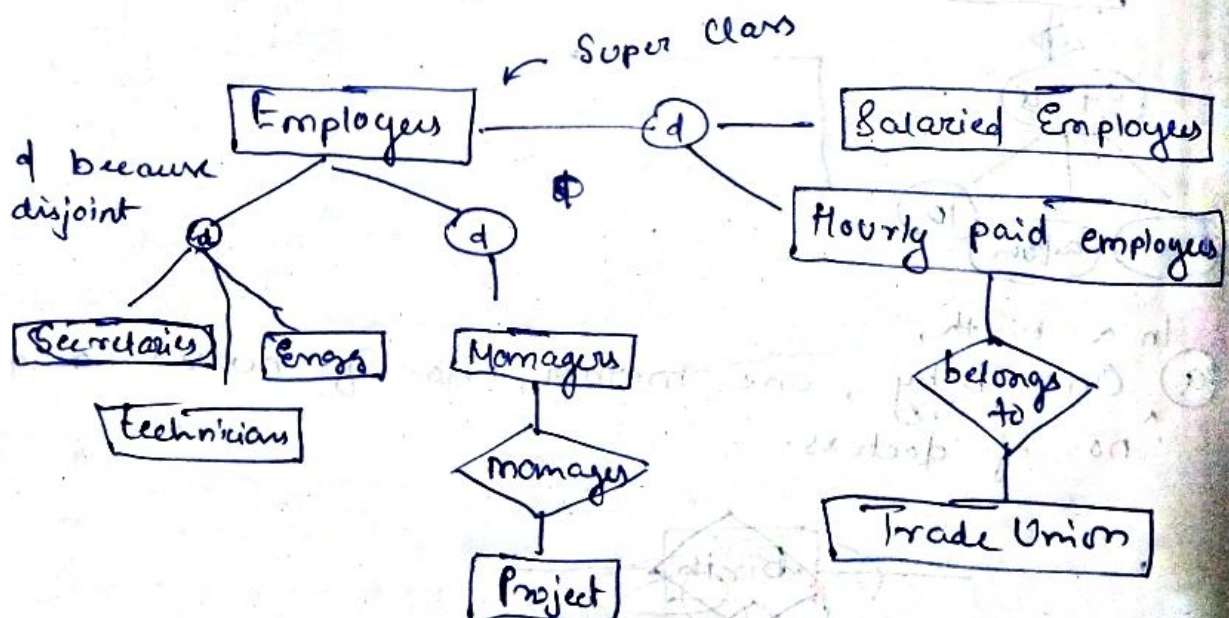
Enhanced Entity Relation Diagram

Class, subclass, inheritance

↓ can be shown using

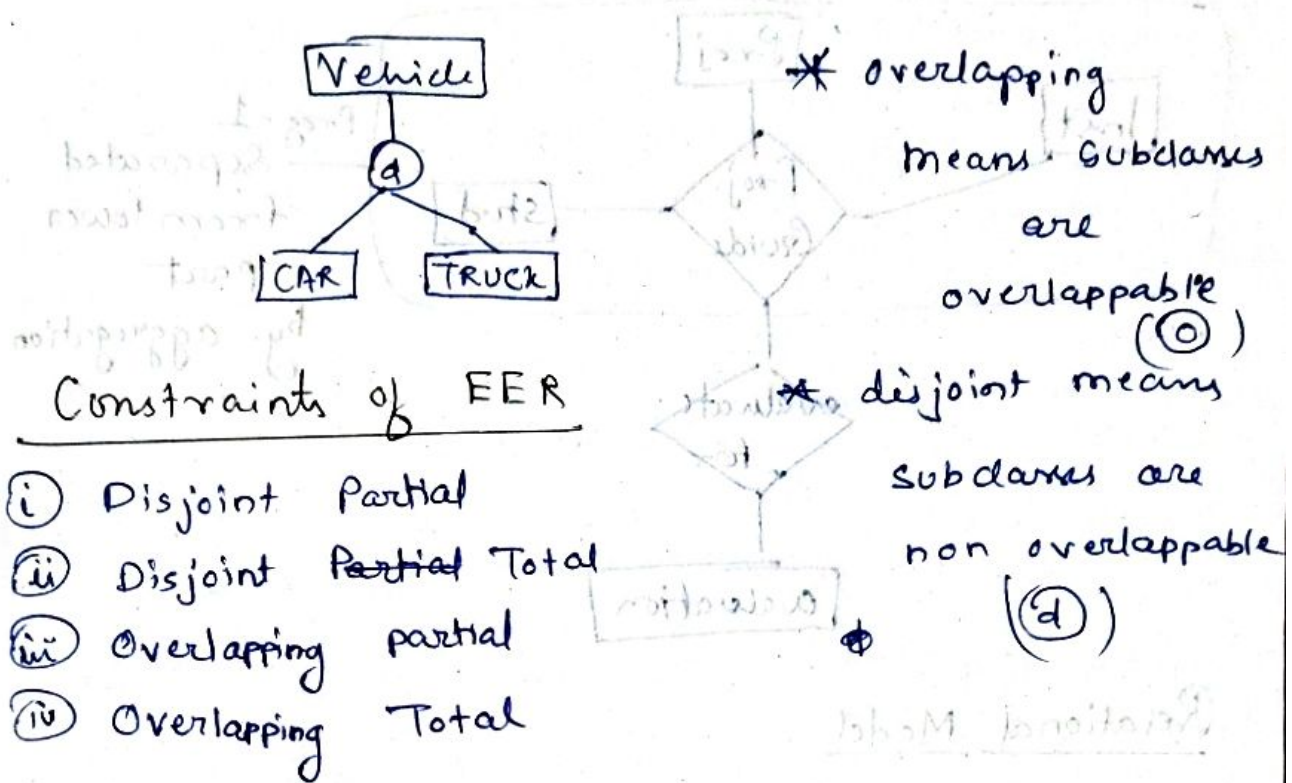
isa, isAND

In a company database
 (En) Employee can be secretaries, technicians, etc.
 Manager manages the projects and the hourly employees belongs to trade union.

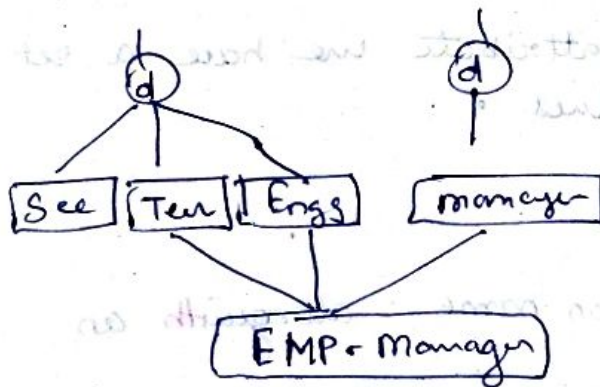


Specialization → giving extra attributes to subclasses other than the inherited ones.

Generalization → Combining subclasses to get Super class.



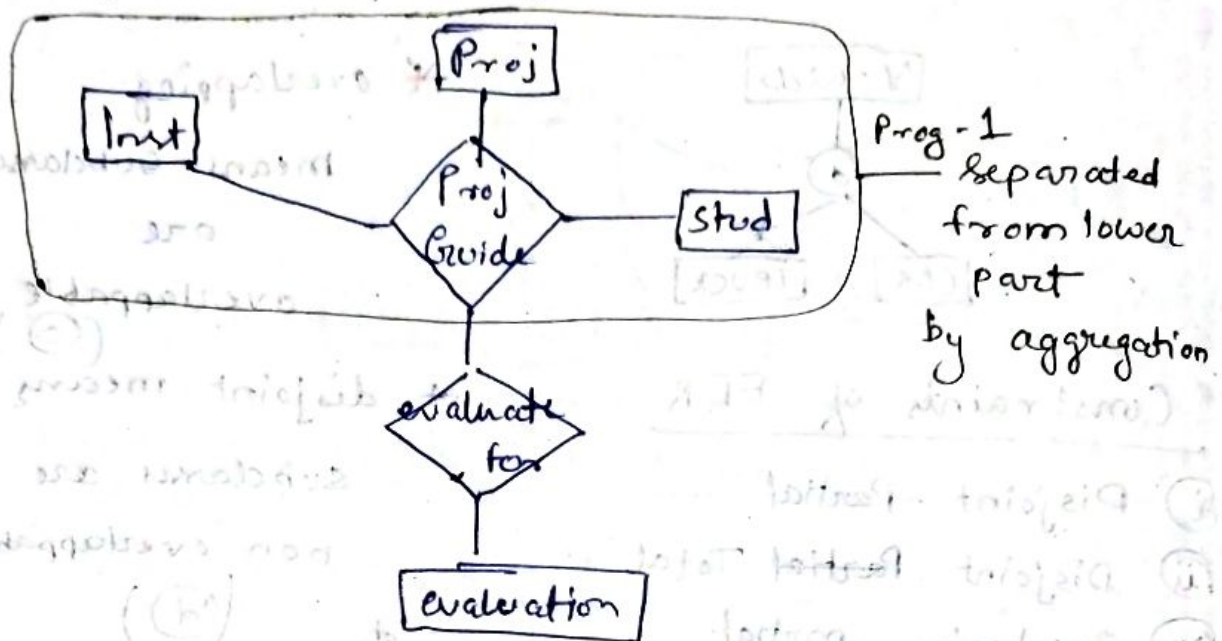
Specialization Lattice
Combining some of subclasses and making a sup another class.



Aggregation

It is an abstraction through which the relationships are treated as higher level entities.

- ⑧ Instructor writes the projects for the students.
Projects evaluated for evaluation.



Relational Model

Each row of relational table is called a tuple.
 Each column in relational table is called an attribute.
Table is called a relation.

Domain :- for each attribute we have a set of all possible values.

Relation Schema

Defined with a relation name along with an attribute.

Student (Name, ID, Address)

Relational Schema

Domain is represented by $\text{dom}(A_i)$ ^{i^{th} Attribute}

Degree $\rightarrow$ The no. of attributes present in the relation.

(Ex) STUD (Name, SSN, Phone no., Address) Degree = 4

$R(A_1, A_2, \dots, A_n)$ Degree = n

$\{A_1\} = \text{Name}$ $\{A_2\} = \text{SSN}$

$r(R)$ $\rightarrow$ R represent a subset of relation schema R having domain $A_1, \dots, A_n$

$\uparrow$
represent relation state of relation schema R

$$r(R) \subseteq (\text{dom}(A_1) \times \text{dom}(A_2) \times \dots \times \text{dom}(A_n))$$

Characteristics of Relation

① Ordering of tuples in a relation

There is no such ordering of tuples in relation

② Ordering of values in a relation (It is very imp.)

Cannot change order of values in relation otherwise there is mismatch of data.

(Ex) ~~Name~~ Stud (Name, SSN, phone)

$\xrightarrow{\text{order}}$

③ $R =$

Relation schema R is a set of attributes then relation state $r(R)$ is a finite set of mapping with $r(R) (t_1, t_2, \dots, t_m)$ where each t_i is a mapping of R to D

$\nwarrow$ domain of all possible values

tuples will get values from the domain of relation schema
tuples can be represented with a set of attributes and values.

Values and Null (When we enter a null value to an attribute)

- * It is to be minimized.
- * This is done when values are unknown or doesn't exist

④ Meaning of a relation → Relation name

Relational Model Notation

* A relation schema $R(A_1, \dots, A_n)$ with degree n can be represented by Q, R, S

* relation state can be represented with q, r, s

* tuples can be represented with t, u, q

* attributes can be represented as RA_i

Key

Student. Age

Should have no redundant values. Needed to identify a table.

Constraints

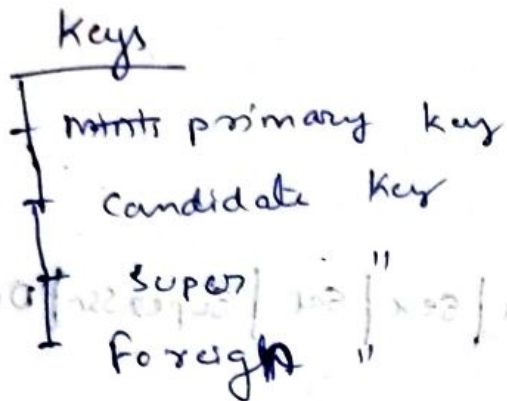
① Two distinct tuples at any state of relation cannot have identical values for the attributes in the key

$t_1 \{ssn\}$ is key (key)

$t_1 \{ssn\} \neq t_2 \{ssn\}$

② Key is to be always a minimal super key.
It is to be designed in such a way that it has minimal attributes.

A key with minimal attributes is called primary key.



Ex) STUD (name, ssn, ID-no, Addr)

can uniquely identify the student

Candidate key

* Any one of the candidate key is called a primary key (we choose)

Super key → Alongwith ssn (a candidate) we took name and joined them and set it as a key. This combination is unique to a student and this key is called super key.

* Super key is not a minimal key.

Foreign key → When we want to establish a relationship b/w 2 tables there should be a common attribute. In this case one key which is common should be primary of one table and foreign of another.

Valid State of the Data

A ~~prec~~ If a database in a state obeys all the integrity integrity of all constraints then it is valid state. Otherwise invalid state.

② Company Database Tables

COMPANY

Employees (Strong Entity)

FN	MN	LN	SSN	BDate	Addr	Sex	Sal	Super SSN	Dno
----	----	----	-----	-------	------	-----	-----	-----------	-----

Department

Dname | Dnum | Mgr-ssn | Mgr-start-date |

Manager - SSN

Department loc

Dnum	Ploc
------	------

Project

Pname | Pnumber | Ploc | ~~P~~ Dnum

Works-on

ESSN | P no. | Hours

1

Employee SSN

Dependent

SSN | dep-Name | sex | Bdate | Relationship

Entity Integrity Constraint

24.2.23 - Fri

Once an attribute is set as primary then no primary key value can be null. Otherwise integrity breaks.

Referential Integrity Constraint

A set of attributes in a relation schema R_1 and ~~relation schema~~ R_2 that references a relation R_2 if it satisfies two conditions.

- # The attributes FK of R_1 is having the same domain as the PK of R_2 .
- # A value FK in tuple t_1 of the relation R_1 , then some tuple t_2 with the same value in relation schema R_2 .

R_1 is referencing relation

R_2 is reference relation.

tuple relational calculus

Relational Algebra and calculus — domain relational calculus

Set of operations to operate on the relational model.

It provides a formal foundation for relational model operations.

Used as a basis for implementing and optimizing the queries in relational database.

Some of its concepts can be used with standard SQL.

Different operators can be used.

① Unary operations

operates on a single relation (table)

① Select (σ)

Selects some of tuples from a relational model satisfying some condition and displays.

Syntax

$$\sigma_{\langle \text{select cond}^n \rangle} (R)$$

Ex) Need to select Depa

Ex) Select the employees with salary 30000

~~Ex) Select~~ salary = 30000 (Employee)

Ex) Select tuples having dept no 4 and salary = 30000
or dept no 5 and salary 50000

$$\sigma_{(Dno=4 \text{ AND } SAL=30000) \text{ OR } (Dno=5 \text{ AND } SAL=50000)} (Employee)$$

$$\sigma_{\langle \text{cond 1} \rangle} (\sigma_{\langle \text{cond 2} \rangle} (R)) = \sigma_{\langle \text{cond 2} \rangle} (\sigma_{\langle \text{cond 1} \rangle} (R))$$

② Cascade of Selection

$$\sigma_{\langle \text{cond 1} \rangle} (\sigma_{\langle \text{cond 2} \rangle} (\dots (\sigma_{\langle \text{cond n} \rangle} (R))))$$

$$= \sigma_{\langle \text{cond 1} \rangle \text{ AND } \langle \text{cond 2} \rangle \dots \text{ AND } \langle \text{cond n} \rangle} (R)$$

iii) PROJECT (π)

Syntax

$$\pi \langle \text{attribute list} \rangle (R)$$

Ex) Display last name, first name, salary of employee

$$\pi \{Lna, Fna, sal\} (Emp)$$

Ex) Display SSN and salary of Employees

$$\pi \{SSN, sal\} (Emp)$$

Sequence of operations & RENAME

Suppose we want to display last name, first name and salary of Dept 4 employees

$$\pi \{LNA, FNA, sal\} (\sigma_{Deptno=4} (Emp))$$

~~Temp~~ OR

$$Temp1 = \sigma_{Deptno=4} (Emp)$$

$$Result = \pi \{LNA, FNA, sal\} (Temp1)$$

iii) RENAME (ρ)

Syntax

$$\rho_s(B_1, B_2, \dots, B_n)(R)$$

new

original

If only attributes
needs changing

$$\rho(B_1, B_2, B_3, \dots, B_n)(R)$$

If only relation

name needs changing

$$\rho(R)$$

Binary Operators

(i) Union

- Ex) Retrieve the SSN of all employees who either work in dept 5 or supervise an employee who work in dept 5

$$\text{DEPT5-EMP} \bowtie \text{Dno} = 5 (\text{Emp})$$

$$\text{RESULT 1} = \pi_{\text{SSN}}(\text{DEPT5-EMP})$$

$$\text{RESULT 1} = \pi_{\text{SSN}}(\text{DEPT5-EMP})$$

$$\text{RESULT 2} = \pi_{\text{SUPER-SSN}}(\text{DEPT5-EMP})$$

$$\text{RESULT} = \text{RESULT 1} \cup \text{RESULT 2}$$

* Degree of both relations needs to be same to union them.

(ii) Intersection

- Ex) Same query as before

(iii) MINUS (-)

$R - S$ = all tuples of R but not in S

$$R \cup S = S \cup R$$

$$R \cap S = S \cap R$$

~~$R - S$~~ $R - S \neq S - R$ (non commutative)

(iv) Cartesian product operator ($\times$)

Also called as cross($\times$) product

$$R \times S$$

By combining each tuple from relation R with tuple from S along with the common tuples

Where $R \times S$
 $n \quad m \leftarrow$ degrees of R and S

result = $n \times m$ relation attributes in resulting relation
 no. of tuples in result table $nR \times mS$

2.3.23 - (The)

② Retrieve a list of names of each female employee's dependents. (those who are dependent on female employees.)

Female-Employees $\leftarrow \sigma_{sex='F'}(EMP)$

Employee-Names $\leftarrow \pi_{FN, LN, SSN}(Female-Employees)$

EMP-DEP $\leftarrow Employee-Names \times Dependents$

$\uparrow$
All dependents

ACTUAL-DEP $\leftarrow \sigma_{SSN = ESSN}$

RESULT $\leftarrow \pi_{FN, LN, Dep-names}(ACTUAL-DEP)$

⑤ Join (X)

If R and S are relations then,

$R \bowtie S$

<Join condition>

Typ.

② Retrieve the name of the manager of each department.

DEPT_MGR = DEPARTMENT $\bowtie$ EMPLOYEE

Mgr-SSN = SSN

RESULT = $\pi_{Dname, LN, FN}(DEPT, MGR)$

Types of Join

① θ -JOIN

EQUI-JOIN $\rightarrow$ If the join condition is equality

② NATURAL JOIN

* The attributes name of both tables must be same. If there are not same but we want to make a natural join then we rename the attribute name.

③ Retrieve the names of the employees who works in research department.

RESE Research-Dept $\leftarrow \sigma_{Dname = 'Research'} (DEPARTMENT)$

DEPT-EMP

RESULT $\leftarrow DEPARTMENT \bowtie_{Dname = Dno} EMPLOYEE$

RESULT $\leftarrow \pi_{FN, LN, DNAME} (RESULT) (DEPT_EMP)$

③ Inner Join

* It is a match and merge operation

1st match with common attribute then merge.

Complete Set of Operations

$\{ \sigma, \pi, \cup, -, \times \}$

$\leftarrow$ Universal Operations

(using these we can find other operations)

(vi) Division

Generally used in query where we

- Q Retrieve the names of employees who work on all projects that 'John Smith' works on.

$$\begin{aligned} \text{SMITH} &\leftarrow \sigma_{\text{FN}='JOHN' \text{ AND } \text{LN}='SMITH'}(\text{EMPLOYEES}) \\ \text{SMITH_PNO} &\leftarrow \pi_{\text{PNO.}} \left(\text{WORKS_ON} \bowtie \text{SMITH} \right) \\ &\quad \text{ESSN} = \text{SSN} \\ \text{SSN_PNO} &\leftarrow \pi_{\text{SSN, PNO}} (\text{WORKS_ON}) \\ \text{SSMS}(\text{SSN}) &\leftarrow \text{SSN_PNO} \div \text{SMITH_PNO} \\ \text{RESULT} &\leftarrow (\text{SSMS} \bowtie \text{EMP}) \\ \text{RESULT} &\leftarrow \pi_{\text{LN, FN}} (\text{SSMS} \bowtie \text{EMP}) \end{aligned}$$

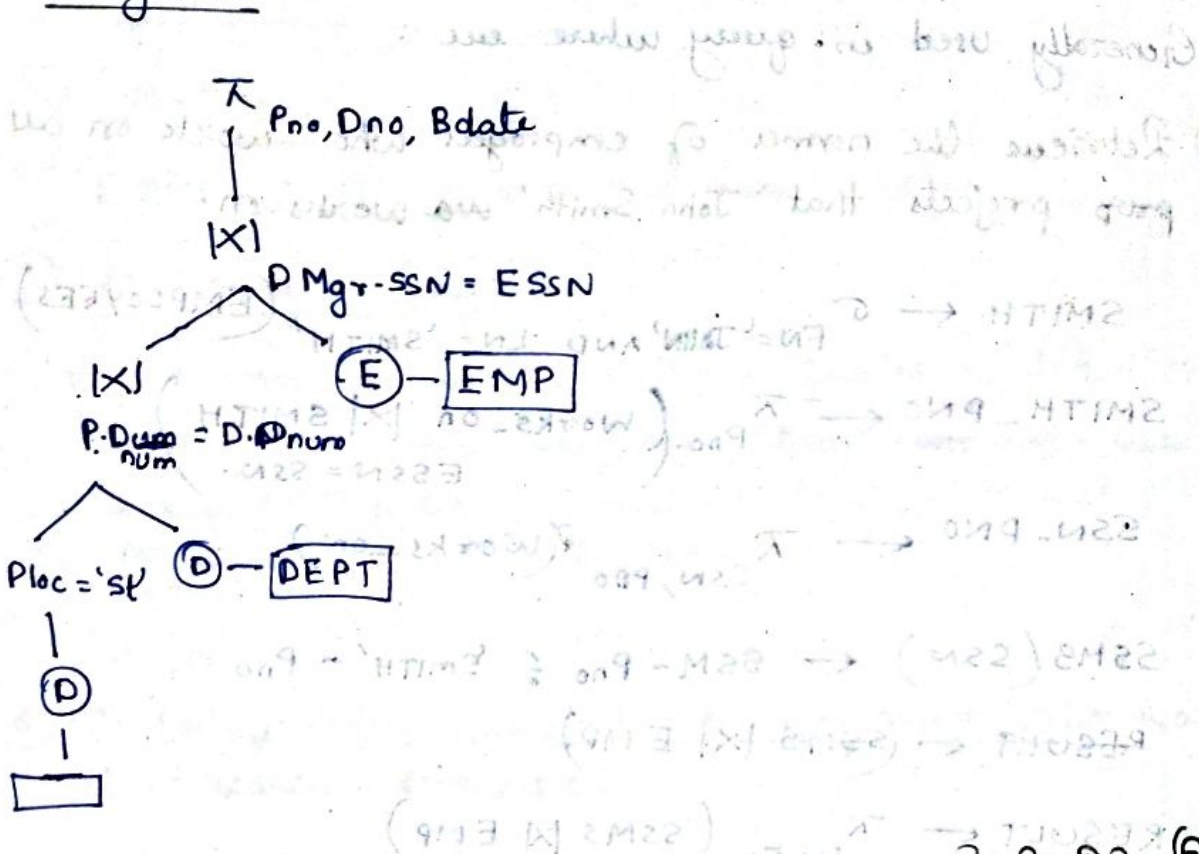
Query Tree

- It is a data structure to represent the relational algebra operations, instead of query writing.
- Evaluation time will be less
- With less time we can solve query.

- Q For every project located in Stamford, retrieve the project no., controlling department no. and department manager's last name, address and Birth date.

$$\begin{aligned} R_1 &\leftarrow \sigma_{\text{Ploc}='Stamford'}(\text{PROJECT}) \\ R_2 &\leftarrow R_1 \bowtie \text{DEPARTMENT} \\ &\quad \text{Dno} \\ R_2 &\leftarrow \sigma_{\text{Pno, Dno}}(R_1) \\ R_3 &\leftarrow R_1 \bowtie R_2 \\ R_4 &\leftarrow \pi \end{aligned}$$

collected by Dr. J. H. M.



3.2.23 - (Fri)

Aggregate Functions & Grouping

↓ → script F

$$\langle \text{grouping attribute} \rangle \mapsto \langle \text{function list} \rangle (R)$$

- ⑥ Group the tables using D.No., count the sum, find the average salary of employees.

D.No Count, Avg-Sal (EMPLOYEE)

Recurring Closure Operation

- ② specify the SSN of ^{all} employees directly supervised the employee whose name is James Borg.

BORG-SSN

FN = 'James' AND LN = 'Borg' (EMPLOYEE)

BORG-SSN $\leftarrow$ $\pi_{SSN}(R_1)$

SUPER-SSN $\leftarrow$ $\pi_{SSN, \text{super-SSN}}(\text{EMPLOYEE})$

RESULT $\leftarrow$ $\pi_{SSN}(BORG-SSN \bowtie SUPER-SSN)$

Outer Join

No loss of information

To avoid the disadvantages of inner join

A set of operations ~~become~~ ^{from} outer join can be used if we want to keep all tuples in R or all the tuples in S or all those in both the relations along with those tuples which are not having the matching values.

Outer join is of 3 types

① Left Outer Join

R $\bowtie$ S

All the tuples of R to be there along with matching values of S

② Right Outer Join

R $\bowtie$ S

All tuples of S along with matches

③ Full Outer Join

R $\bowtie$ S All tuples of S along and R

- ② Display the name and address of all employees who work for the research department.

$DNAME \leftarrow \sigma_{DNAME='Research'}(DEPT)$
 $DNAME_EMP \leftarrow DNAME \bowtie EMP$
 $Dnum = Dno$
 $Dno = Dnum$
 $RESULT \leftarrow \pi_{FN, LN, Address}(DNAME_EMP)$

- ③ List the names of all employees with two or more dependences.

$T1 \leftarrow \sigma_{(DEPEN)}$
 $T2 \leftarrow \sigma_{NO. of DEPEN \geq 2}(T1)$
 $T3 \leftarrow \pi_{FN, LN}(T2 \bowtie EMP)$
 $ESSN = SSN$

ASSIGNMENT

Coorh
Navadev
Bipin Desai

All g algebra
questions in
assignment
copy

Tuple Relational Calculus

$\{t \mid COND(t)\}$
 $\downarrow$
 resultant tuples to be displayed

$\rightarrow$ satisfying.
 $\rightarrow$ + to be displayed satisfied this condition.

$RESULT = \pi_{FN, LN}(\sigma_{Salary > 30000}(EMP)) \Rightarrow$ In Algebra

$\{t \mid EMP(t) \text{ AND } t.salary > 30000\} \Rightarrow$ In Tuple Relational Calculus.

- ④ Display the BDate and Address of the employees 'John B Smith'

Rel Algebra

RESULT = $\pi_{Bdate, Address} (\sigma_{FN='John', LN='SMITH', MN='B'} (EMP))$

Calculus

$\{ t.Address, t.Bdate \mid EMP(t) \text{ AND } t.FN='John' \text{ AND } t.MN='m' \text{ AND } t.LN='Smith' \}$

② Display the name and address of employees of research department.

Calculus

$\{ t.FN, t.LN, t.Address \mid EMP(t) \text{ AND } DEPT(t)(\exists d) \text{ DEPT}(d) \text{ AND } d.Dname='Research' \text{ AND } d.Dname=t.dno \}$

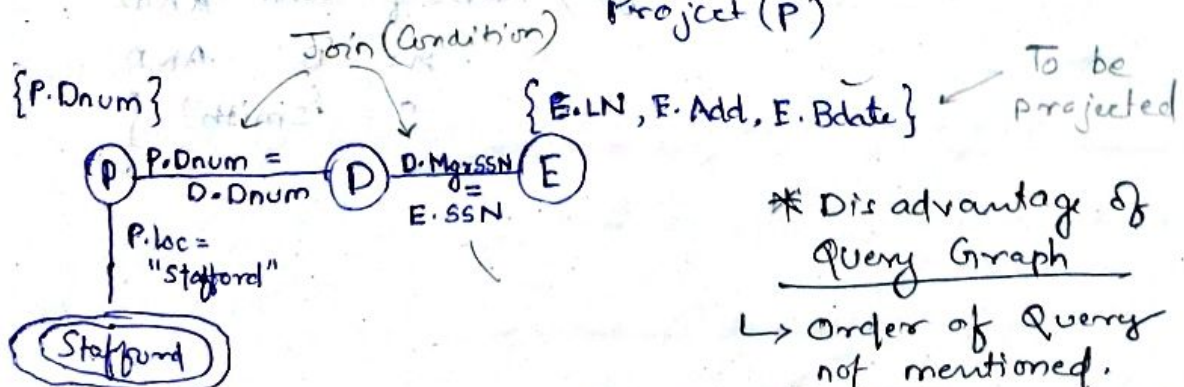
16.3.23 - (Thu)

Query Graph

- * Relations to be represented by Circles
- * Edges represent conditions
- * Const value represented by 
- * attributes to be retrieved for respective relation, then they are to be displayed above relation.

② Display the LN, Address, Bdate of managers working in the project location 'Stafford'.

Relations to be used $\rightarrow$ Employees (E), Department (D), Project (P)



* Disadvantage of Query Graph

$\rightarrow$ Order of Query not mentioned.

* Efficient way of Query processing is Query Tree.

Safe Expression

If output is finite then expression is safe, otherwise unsafe (infinite o/p)

② $\{t \mid \text{NOT}(\text{EMP}(t))\} \Rightarrow$ displays all tuples except

* This is to be avoided. Employees (infinite) So Unsafe Expression

Domain Relational Calculus (Order of Attribute is important)

General Conditions

$\{x_1, x_2, \dots, x_n \mid \text{COND}(x_1, x_2, \dots, x_n) \wedge x_{n+2} \dots x_{nm}\}$

attribute condition

② Retrieve the Bdate and address of employee

John B Smith
Relational Algebra

$R \leftarrow \pi_{\text{Bdate, Address}} (\sigma_{\text{FN='John' AND MN='B' AND LN='Smith'}} (\text{EMPLOYEES}))$

Domain Relational Calculus (1st Way)

$\text{EMP}(qrstuvwxyz)$

$u = \text{Bdate}$

$v = \text{Address}$

$\{uv \mid (\exists q)(\exists r)(\exists s) \dots (\exists z) (\text{EMP}(qrstuvwxyz) \text{ AND } q = \text{'John'} \text{ AND } r = \text{'B'} \text{ AND } s = \text{'Smith'})\}$

- ④ Retrieve the name and address of all employees who work for the research department.

Relational

$$R1 \leftarrow \sigma_{Dname = 'Research'} (DEPT)$$

$$R2 \leftarrow EMP \bowtie R1$$

$$E.Dno = R1.Dno$$

$$R \leftarrow \pi_{FN, MN, LN, Add} (R2)$$

Domain Relational Calculus (2nd way)

$$EMP(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, x_{10}) \quad DEPT(l, m, n, o)$$

$$\{ x_5 \mid \exists x_2 \exists x_3 \exists x_4 \exists x_6 \exists x_7 \exists x_8 \exists x_9 \exists x_{10} \exists l \exists m \exists n \exists o$$

$$EMP(x_1, \dots, x_{10}) \text{ AND } (DEPT(l, \dots, o))$$

$$\text{AND } l = 'Research' \text{ AND } m = z$$

$$l = D.Dno$$

$$z = E.Dno$$

$$m = D.Dno$$

- ④ Retrieve ~~For a project~~ List the project no., controlling Dno, Dept manager, LN, Bdate, Address for a project at 'Stafford'.

$$EMP(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, x_{10}) \quad DEPT(l, m, n, o) \quad PROJECT(a, b, c, d)$$

$$\{$$

Relational Database Design using ER-to-Relational Model

① List the names of employees who have no dependents.

② List the names of the managers who have 2 dependents.

Relatio

Relational Database Design using ER-to-Relational Model Algorithms

① Mapping of Regular Entity types

Conversion of ER to relational for regular entity types.

② Mapping of Weak Entity types

Weak Entity = DEPENDENT

ESSN	DEPARTMENT

③ Mapping of Binary 1 to 1 relationship types

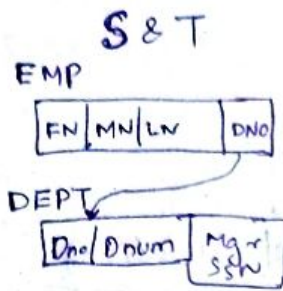
(i) Foreign key Approach

(ii) Merged Reference Approach

(iii) Cross Reference Approach

(i) Foreign Key Approach

23.3.23 - (Thu)



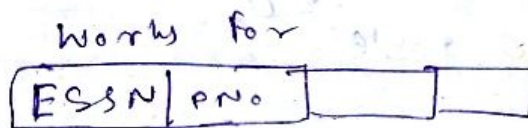
* To join 1 to 1 relationship types we take 1 key from a relation as foreign key and joint it with primary key of other table

(ii) Merged relationship approach

Make a new table and join the relations and store. This method makes the table very big and queries are slow.

(iii) Cross Reference or relationship relation approach.

To show 1 to 1 relationship, take primary key of "employer" and take P.K of "works for"



↑
To show relationship

(4) Mapping of 1 to n relationship

* Suppose R is the relation with n set of relations.

Include as foreign key in S the primary key of relation P that represents the other entity types participating in R .

② insert Dno. of Dept in Emp relation as foreign key

* Suppose supervisor supervises the ordinates.

Here both relations are same so it is a recursive relationship type. (NOTHING TO BE CHANGED)

⑤ Mapping of Many-to-Many

Suppose Emp works on projects
many - to - many

⑥ How to map Multivalued Multivalued relations

How to map depart loc. of TCS

Take a new table ^{named} with Dept-Location.

Dnum	Dloc
------	------

⑦ Mapping of Non-binary relations

Suppose supplier supply supplies parts to projects

Supplier Parts Projects

Create a table named supply.

Add all PK ^{of tables} to new table

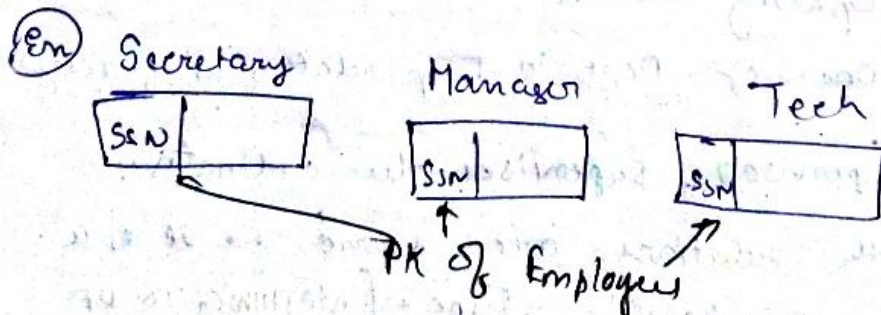
SUPPLY

SNo	PNo	Proj No.
-----	-----	----------

⑧ Mapping of ^{Specialization} Generalization and ^{Generalization} Specialization

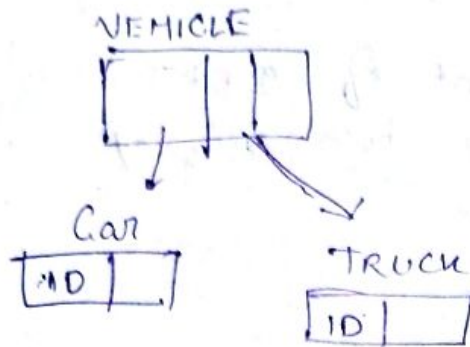
~~Specialization~~ → Employees categories

RULE Create individual table for each specialization.



Generalization :-

Create a new table



Database Design

* A good design means least query time.

DB Design is of 2 types

- Bottom UP
- Top Down :- Design by analysis

Bottom-UP :- Design by Synthesis

↳ Generally it is not in use

Informal

Design guidelines for ~~static~~ relation schema

- Semantics of attribute ^{meaning of the attribute}
- Reducing Redundant ^{Superior Tuples}
- Reducing NULL values
- Disallowing Superior tuples

* Name and Semantics are important.

* Try to store less no. of duplicate tuples

Anomalies

① Insertion Anomaly

Created a table EMP-DEPT

We have EMP info but not of DEPT
because Employees have not been assigned
DEPTS.

② Functional Dependency

It is a constant b/w two sets of attributes
from the database.

$$X \rightarrow Y \quad (X \text{ functionally determines } Y)$$

This can say that for any tuples t_1 and t_2

$$R \rightarrow$$

If t_1 and t_2 are tuples then

$$t_1[x] = t_2[x]$$

$$t_1 \neq t_2 \text{ in } R$$

$$t_1[x] = t_2[x]$$

$$\text{then } t_1[y] = t_2[y]$$

③ ~~SSN~~ functionally de

$$SSN \rightarrow Ename$$

$$\text{if } t_1[SSN] = Ename[SSN]$$

$$\text{Suppose } Pnum \rightarrow \{Pname, Pbc\}$$

trivial

$$X \rightarrow Y \text{ if } X \supseteq Y$$

* The dependency which is not trivial we call it non trivial.

Closure of Attribute (X^+), where x is attribute

$$X^+ F = \left\{ \begin{array}{l} \text{ssn} \rightarrow \text{Ename}, \\ \text{Pnum} \rightarrow \left\{ \begin{array}{l} \text{Pname} \\ \text{Ploc} \end{array} \right\} \end{array} \right\}$$

$$\left\{ \text{ssn}, \text{Pnum} \right\} \rightarrow \text{Hours}$$

Closure of functional dependencies

$$\left\{ \text{ssn} \rightarrow \text{ssn} \right\} \text{Ename}$$

$$\left\{ \text{ssn} \rightarrow \text{Ename} \right\}$$

$$\left\{ \text{Pnum} \rightarrow \text{Pnum} \right\}$$

$$\left\{ \text{Pnum} \rightarrow \text{Pname} \right\}$$

$$\left\{ \text{Pnum} \rightarrow \text{Ploc} \right\}$$

$$\left\{ \left\{ \text{ssn}, \text{Pnum} \right\} \rightarrow \text{Hours} \right\}$$

$$\left\{ \left\{ \text{ssn}, \text{Pnum} \right\} \rightarrow \left\{ \text{ssn}, \text{Pnum} \right\} \right\}$$

Closure of attribute

$$\text{ssn}^+ = \left\{ \text{ssn}, \text{Ename} \right\}$$

$$\text{Pnum}^+ = \left\{ \text{Pnum}, \text{Pname}, \text{Ploc} \right\}$$

$$\left\{ \text{ssn}, \text{Pnum} \right\}^+ = \left\{ \left\{ \text{ssn}, \text{Pnum} \right\}, \text{Hours}, \text{Ename}, \text{ssn}, \text{Pnum}, \text{Pname}, \text{Ploc}, \text{Hours} \right\}$$

* This is known as attribute preservation.

Q Sno $\rightarrow$ Sname

Cno $\rightarrow$ Cname

$$\{Sno, Cno\}^+ = \{ \{Sno, Cno\}, Sno, Sname, Cno, Cname \}$$

Q $A \rightarrow B$ $BC \rightarrow D$ $C \rightarrow E$ $CD \rightarrow F$

$$(A, C)^+$$

$$(A, C) = (A, C), B, A,$$

Q $R = \{A, B, C, G, H, I\}$

$$F = \left\{ \begin{array}{ll} A \rightarrow B & CG \rightarrow I \\ A \rightarrow C & B \rightarrow H \\ G \rightarrow H & \end{array} \right\}$$

F^+

Equivalence of sets of FD

Cover $\rightarrow$ A set of FDs (F) is said to cover another set of FDs (E) if E is in F^+

Equivalence $\rightarrow$ Two sets of FDs are equivalent if $F^+ = E^+$. If every FD of E can be derived from F^+ and vice-versa.

Minimal Cover

A minimal cover of a set of FDs F that satisfy the property that every dependency in E is in the closure of F and the following conditions are satisfied.

- * Every FD of E should be in F^+
- * Every FD in F has single attribute in RHS

* We cannot replace $x \rightarrow A$ in F such that $y \rightarrow A$ $y \subseteq x$

when

* We cannot remove any dependency from F . Then we say it has minimal cover.

Then

Q $F = \{B \rightarrow A, D \rightarrow A, AB \rightarrow D\}$ find minimal cover.

$BB \rightarrow AB$ [Augmentation]

$B \rightarrow AB, AD \rightarrow D$ [Transitive]

$B \rightarrow D, D \rightarrow A$

$B \rightarrow A$

[Transitive]

↑
minimal cover of F

Normal Form

Process of decomposing tables to optimize query-time.

* Lossy Join — lossy (if loss is there)

* Dependency Preservation. (all dependencies are to be preserved.)

1NF

All the attributes are to be with atomic values.

Suppose, $ssn / Ename / Bdate$ } 1NF (Table has atomic values)

Suppose, $Dname / Dnum / Dmgr-ssn / Dloc$ Not in 1NF

because $Dloc$ has multi-valued

(Dept can be at 71 loc)

Normalization

Drum | Drum | Drums.en | Dial1 | Dial2 | Dial3

In this method, if dept is at one loc, then rest are here so this is to be avoided (not a correct way)

The correct solution is,

To decompose main table into diffent tables such that there is no ~~loss~~ loss of data or spurious tuples after segrating.

3.4.23 - Mon

2 NF

Condⁿ → All the attributes should be ~~with~~ atomic values, then relation is
↓
no multivalued attribute or further dividing possible

2NF

Full Functional Dependency :-

- Suppose we have $X \rightarrow Y$, we can say it is full functionally dependancy, if there not exists $x \rightarrow z$ (in short no transitive relation)

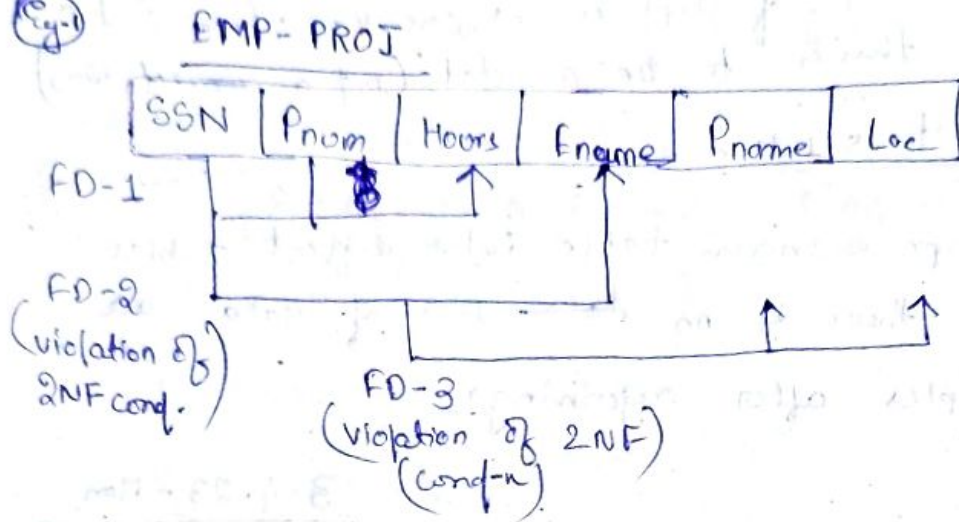
- A functional dependency $x \rightarrow y$ is full functional dep. if ^{on} removal of any attribute from x there will be no dependancy.

- If the above is true then it is partial functional dependancy.

Ex { SSN, Pnum $\rightarrow$ house (full functⁿ dependancy)
{ SSN, Pnum $\rightarrow$ Enum (Partial functional dependancy)
Pnum $\rightarrow$ Enum

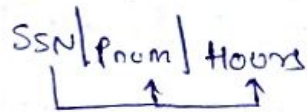
* A relation schema R is in 2NF in every non-prime attribute in A in R is fully functionally dependent on primary key of R .

Ex-1

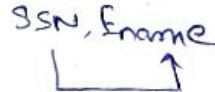


• Now we subdivided the table to make them satisfy the 2nd NF. Condⁿ

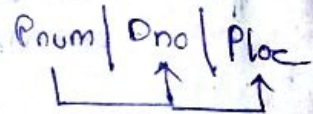
R_1



R_2



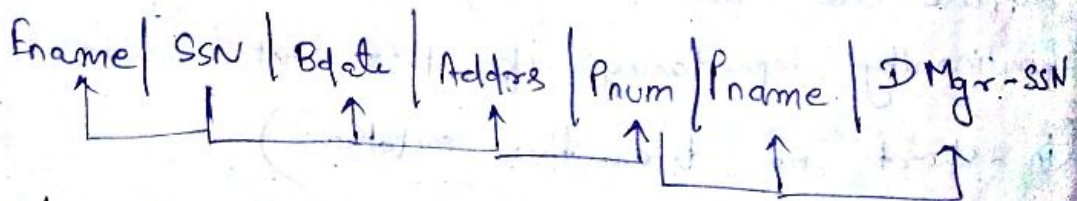
R_3



(Here dependancy is preserved)

Ex-2

EMP-DEPT



Transitive Dependency

A FD $x \rightarrow y$ in relational schema R is transitive dep. if there exists a set of attributes z such that z is not a candidate key and we have functional dep. $x \rightarrow z$ and $z \rightarrow y$.

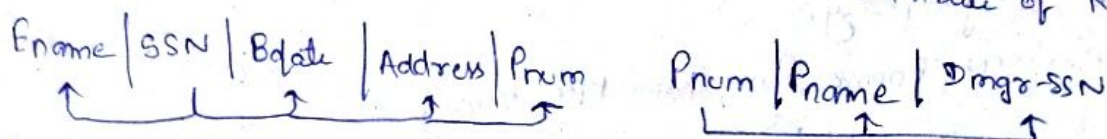
3NF

- A relational schema is in 3NF if it satisfies 2NF and there are no non prime attribute of R transitively dependant on primary key.

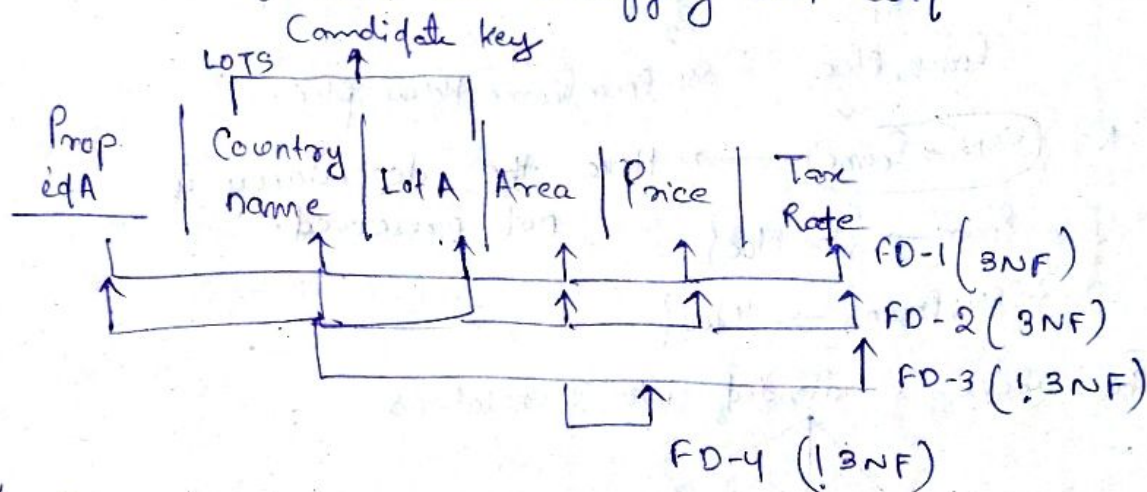
- A relation schema R is in 3NF whenever a non-trivial $x \rightarrow A$ holds in R either (i) x is super key OR

(Eg) Refer (Eg-2)

(ii) A is a prime attribute of R

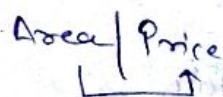
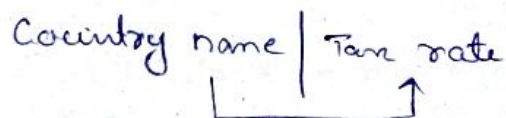


Now both of them are satisfying 3NF condⁿ



To preserve the functⁿ dependancy and satisfy 3NF condition,

Take the FD-1 and FD-2 as the original table and FD-3 and FD-4 are represented as:-



BCNF (Boyce-Codd Normal Form)

For any non-trivial FD $x \rightarrow A$ if x is the superkey of R.

(Eg) SUPPLY (S#, P#, Sname, Scontrol, Price)

S# = ~~Sno~~ Sno

S# = Scontrol

S# P# → Price

(It satisfies BCNF)

Now, we can divide the table.

$(S\#, Sname)$ and $(S\#, Sschool)$

Now, they satisfy BCNF

⑧ $R(title, Perfor, Style, Price, Label, Producer)$

$title \rightarrow lab, style. \quad S + y = Price.$

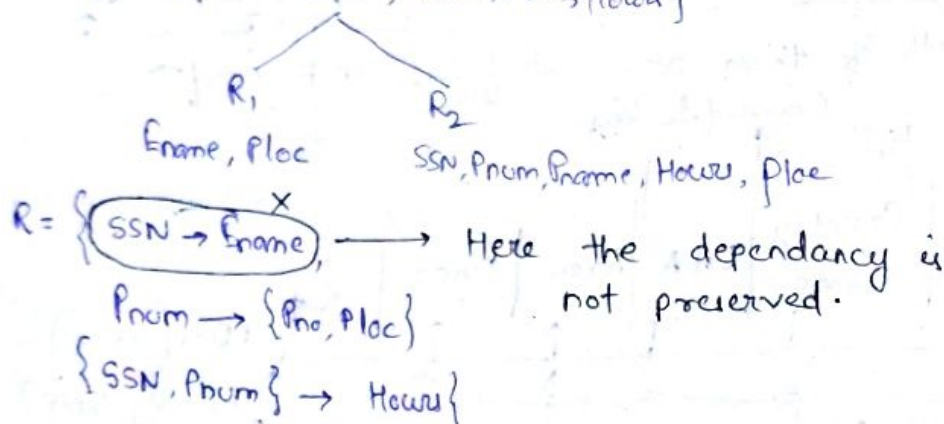
$Perfor \rightarrow Producer.$

Dependency Preservation

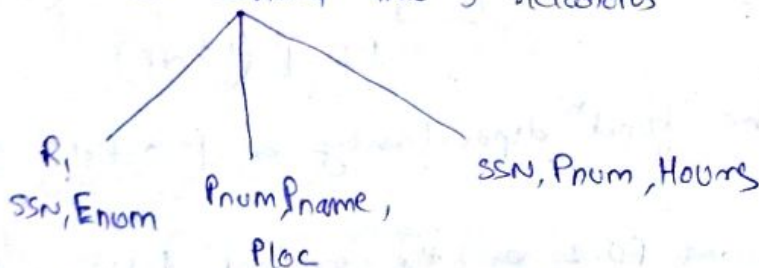
6.4.23 - (Thu)

It is a property.

If $R = \{SSN, Ename, Pnum, Pname, Ploc, Hours\}$



The above is divided into 3 relations



In this division, all the dependencies are preserved.

~~Lossy decomposition~~

Loss-less Join

Dependency $D = \{R_1, R_2\}$ of R has a lossless join property w.r.t a set of FD on R if either of the two conditions is satisfied.

$\rightarrow FD(R_1, R_2) \rightarrow (R_1 - R_2) \text{ is in } F^+$

$\rightarrow FD(R, R_2) \rightarrow (R_2 - R_1) \text{ is in } F^+$

If we refer the previous example in dependency preservation,

$$\{SSN, Ename, Pnum, Pname, Ploc, Houri\} \rightarrow Ename, Ploc, Pnum$$

$$\{SSN, Ename, Pnum, Houri, Pname, Ploc\} \rightarrow SSN, Pnum, Ename, Houri, Ploc$$

$$R \cap R_2 = Ploc \rightarrow Ename, Ploc$$

$$Ploc \rightarrow SSN, Pnum, Pname, Houri, Ploc.$$

$$F^+$$

$$\left\{ \begin{array}{l} SSN \rightarrow SSN, \\ SSN \rightarrow Ename, \\ Pnum \rightarrow Pnum, \\ Pnum \rightarrow Pname, Ploc, \\ SSN, Pnum \rightarrow SSN, Pnum, \\ SSN, Pnum \rightarrow Houri \end{array} \right\}$$

Multivalued Dependency

If $x \twoheadrightarrow y$ specified on relation schema R where x and y are subsets of R which specifies the constraint in relation state r of R .

If two tuples t_1, t_2 in R such that $t_1[x] = t_2[x]$ then there are other two tuples t_3 and t_4 in R which satisfies the following properties.

$$Z = (R - (x \cup y))$$

(Eg.) If $R = \{Ename, SSN, Pname, Ploc\}$ $x = SSN$ $y = Pname$
 If this, then $z = Ename, Ploc$

(i) $t_3[x] = t_4[x], t_1[y] = t_2[y]$

(ii) $t_3[y] = t_1[y], t_4[y] = t_2[y]$

(iii) $t_3[z] = t_2[z], t_4[z] = t_1[z]$

(Eg) $Ename \twoheadrightarrow Pname$

It means employees work in different departments

Trivial Multivalued Dependency

Multivalued Dependency $x \twoheadrightarrow Y$ in R if

(a) Y is a subset of X

(b) $X \cup Y = R$

OR

If either of condition is satisfied, it is trivial dependency or non-trivial dependency

Inference rule for MVD and FD:-

IR_1 (Reflexive), if $X \supseteq Y$, then $X \rightarrow Y$ (X functionally determines Y)

IR_2 (Augmentation), $\{X \rightarrow Y\} F \quad XZ \rightarrow YZ$

IR_3 (Transitive), $\{X \rightarrow Y, Y \rightarrow Z\} F \quad X \rightarrow Z$

IR_4 (Complementation Rule) $\{X \rightarrow Y\} F \quad X \rightarrow (R - (X \cup Y))$

IR_5 () $\{X \rightarrow Y\}$ and $W \supseteq Z$
 $WX \rightarrow YZ$

IR_6 (Transitive rule for MVD) $\{X \rightarrow Y\} \{Y \rightarrow Z\} F \quad X \twoheadrightarrow (Z - Y)$

IR_7 (Replication rule) $\{X \rightarrow Y\} F \quad X \rightarrow Y$

IR_8 () $\{X \rightarrow Y\}$ and there exists W

with property (i) $W \cap Y$ is empty

(ii) $W \rightarrow Z$ (W functionally determines Z)

(iii) $Y \supseteq Z$ then $X \rightarrow Z$

4NF (4th Normal Form)

A relation schema R is in 4NF with FD if and here F (FDs & MVDs) for non-trivial MVD $X \twoheadrightarrow Y$ in F^+ , X is a superkey of R .

Eg Emp
 $\text{Empname} | \text{Prname} | \text{Dname}$

$\text{Empname} \rightarrow \text{Prname}$
 $\text{Empname} \rightarrow \text{Dname}$

It is not in 4th NF. Now, we divide it into two

$\text{Empname} | \text{Prname}$

$\text{Empname} | \text{Dname}$

Now, it satisfies 4NF condition.

~~Non additive join Decomposition into 4NF:~~

Non additive join Decomposition into 4NF:

Let relation schema R in R_1 and R_2 , where F is a set of FD in MVD. If it satisfies the following conditions:-

(i) $R_1, R_2 \rightarrow (R_1 - R_2)$

(ii) $R_1, R_2 \rightarrow (R_2 - R_1) \quad \boxed{OR}$

5NF and Join Dependency:-

Join dependency of $R_1, R_2, \dots, R_n$, if r is the relation state of R , then we can say it is join decomposition of $(\pi_{R_1}(r), \pi_{R_2}(r), \dots, \pi_{R_n}(r)) = r$

- MVD is special case of join dependency.
- A join dependency of the above form is trivial if one of the relation schema R in JD is equal to R .

$$JD(R_1, R_2, R_3, \dots, R_n) = R$$

Join dependency

- A relation schema is in 5NF or project join normal form w.r.t. set of F (it is a set of FD, MVD, JD), if for every non-trivial JD in F^+ , then R is a super key of R .

Eg) SUPPLY

Sname | Part name | Proj. name

Suppose we have divided it into two

$R_1 = (\text{Sname}, \text{Part name})$

$R_2 = (\text{Sname}, \text{Proj-name})$

$R_3 = (\text{Part-name}, \text{Proj-name})$

It is a join decomposition and all the JD is 5NF.

10.4.23 - (Mon)

Query Processing and Optimization

- To reduce processing time $\rightarrow$ optimization is required.

Query is a high level language (Ex:- SQL)

↓
Scanning, parsing and validation (dividing Query)

↓
immediate form of query

↓
Query optimizer (software)

↓
Execution plan

↓
Query code generator

↓
Code to execute the query

↓
runtime database processor

↓
result of query

Algorithm for SELECT & JOIN

- ① $\sigma_{SSN = '1234'} (EMP)$
- ② $\sigma_{Dno > 6} (EMP)$
- ③ $\sigma_{Dno = 5} (EMP)$
- ④ $\sigma_{Dno = 5 \text{ AND } SAL > 30000 \text{ AND } SEX = 'F'} (EMP)$
(AND HOURS WORKS-ON)
- ⑤ $\sigma_{ESSN = '1234' \text{ AND } Dno = 10}$
(OR)

Ways of Searching

① Linear Search:-

② Binary Search:-

- More efficient than linear search
- Divide the total set of records find the mid value
- Assumption:- records are arranged in increasing order.

Example :-

1000 } contains 1234
1500 → mid value
2000 }

→ 1000 } contains 1234
1250
1500

and so on...

• Division operation takes more time.

③ Using a Primary Index or Hash Key:-

↓
Mathematical function to get address. We can directly go to the record without doing any searching.

- $\{, \{, \}, \} =$ If there are the conditions we can get multiple records.

④ SS:-

- Using a clustering index to retrieve multiple records.

⑤ SG:-

- Using a secondary index on an equality comparison.

Ex:- $30000 \leq SAL \leq 35000$

- Selection is with secondary Index
- It is a range query

⑥ S7:-

- Conjunctive selection using secondary Index

Ex:- Query 5

⑦ S8:-

- Conjunctive selection using secondary Index

Ex:- Query 4

⑧ S9:-

- Query with both conjunction and disjunction

Ex) Query 4 $\times$ Query 5

Implementing Join

EMP |X| DEPT

Dno = Dnum

DEPT |X| EMP

Mgr-ssn = ssn

Join algorithms can be of 3 types:-

① Nested Loop Join:-

$R |X| S$

Suppose t is tuple in R (outer loop)
 s is tuple in S (inner loop)

$t\{A\} = s\{B\}$ (tuple to be selected)

A, B are common attribute
 $R \quad S$

② Single loop Join:-

$s\{B\} = t\{A\}$

• first find out $s\{B\}$ then compare it with $t\{A\}$

③ Sort merge Join:-

• Sort the records of R & S with the joining condition.

Method to Optimize the Query Process

* Heuristic Optimization

① Select the last name of EMP who

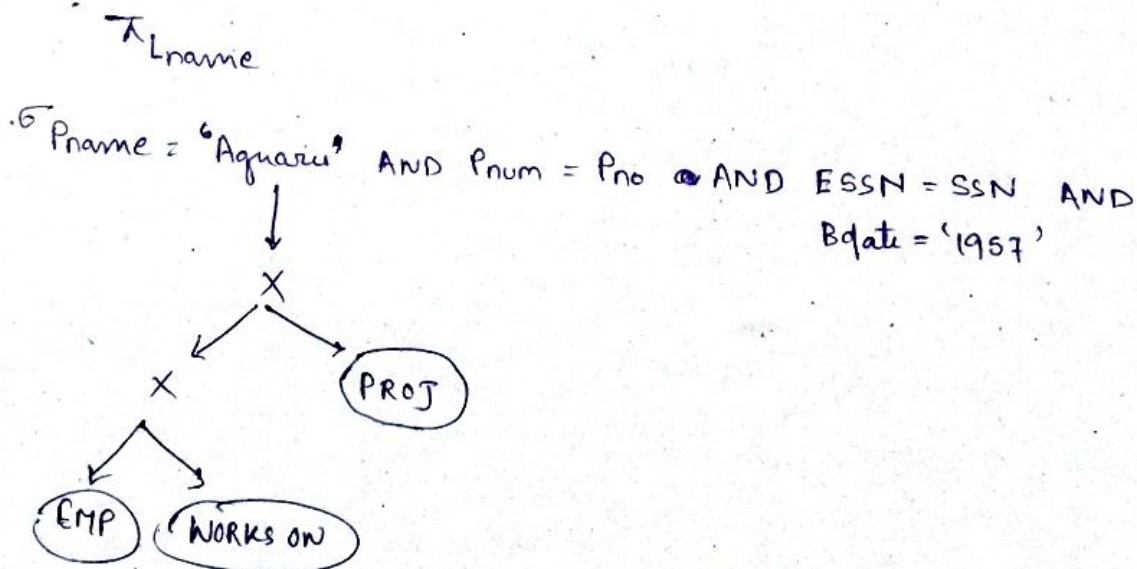
WORKS-ON Proj Name - Aquarius having Bdate > 1957.12.31

Query:-

$R_1 \rightarrow \sigma_{Pname = 'Aquarius' (PROJ) \mid (WORKS ON)}$
 $Pnum = Pno$

$\pi_{Lname} (R_1 \mid \sigma_{Bdate = '1957' (EMP)})$
 $ESSN = SSN$

Query Tree:-



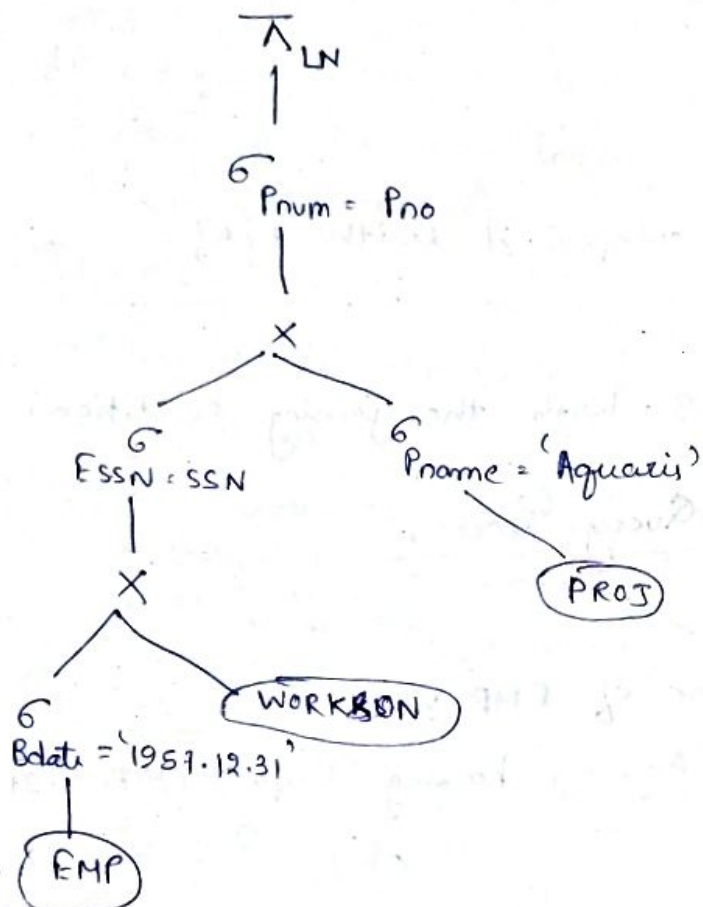
Canonical form of the Query

Joining = Selection & Cartesian Product

Step-2

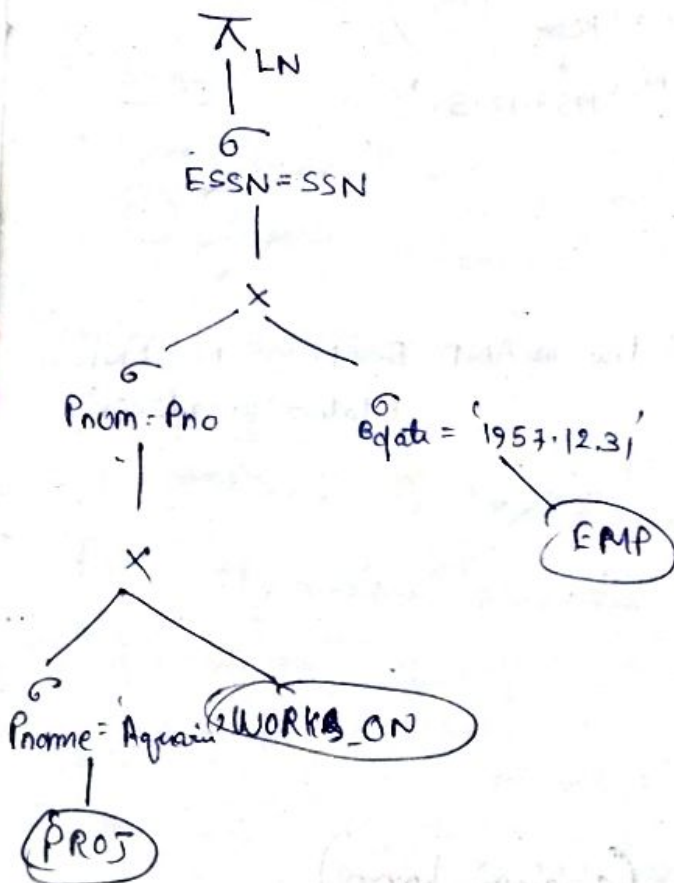
Moving selection down the query tree

13.4.23 - (Thu)



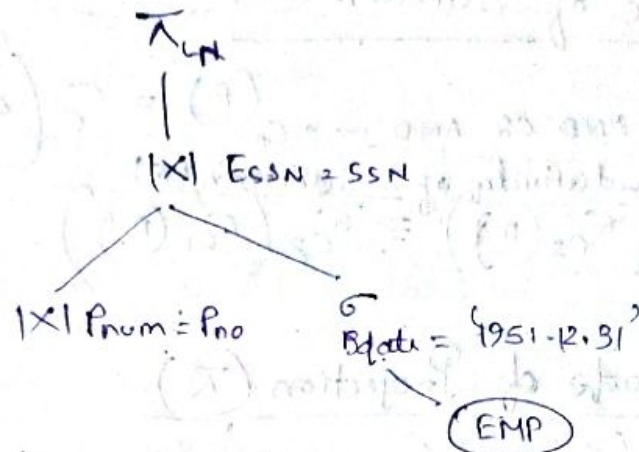
Step-3

Applying the most restrictive selection first



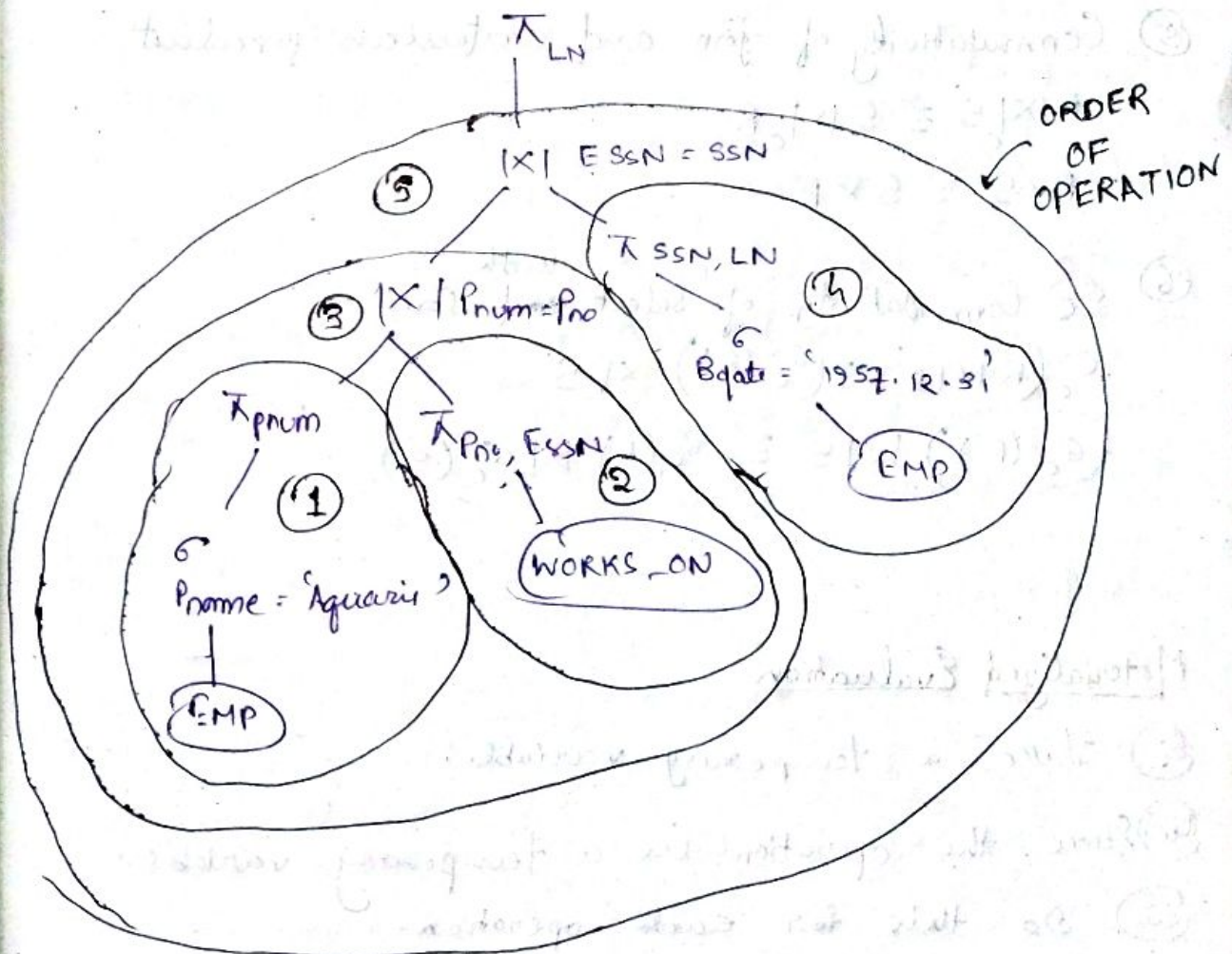
Step-4

Replacing Cartesian Product and Selection with a join.



Step-5

Moving the projection operation down the query tree



So these are the steps for query optimizations.

Generalised Transaction Rules for Relational Algebra Operation.

① Cascade of Selection (σ)

$$\sigma_{C_1 \text{ AND } C_2 \text{ AND } \dots \text{ AND } C_n}(R) = \sigma_{C_1}(\sigma_{C_2}(\sigma_{C_3} \dots (\sigma_{C_n}(R))))$$

② Commutativity of Selection (σ)

$$\sigma_{C_1}(\sigma_{C_2}(R)) \equiv \sigma_{C_2}(\sigma_{C_1}(R))$$

③ Cascade of Projection (π)

$$\pi_{List1}(\pi_{List2}(\dots (\pi_{Listn}(R)))) = \pi_{List1}(R)$$

If all list are sub lists of list 1

④ Commutativity of Projection with Selection

$$\pi_{A_1, \dots, A_n}(\sigma_C(R)) \equiv \sigma_C(\pi_{A_1, A_2, \dots, A_n}(R))$$

⑤ Commutativity of join and cartesian product

$$R \bowtie S \equiv S \bowtie R$$

$$R \times S \equiv S \times R$$

⑥ Commutativity of Select and Join

$$\sigma_C(R \bowtie S) \equiv (\sigma_C(R)) \bowtie S$$

$$(\sigma_C(R)) \bowtie S \equiv \sigma_{C_1}(R) \bowtie \sigma_{C_2}(S)$$

Materialized Evaluation

(i) Take a temporary variable.

(ii) Store the operation in a temporary variable.

(iii) Do this for each operation.

(iv) Finally join them all.

Pipelined Evaluation

⑦ Commutativity of join with Cartesian Product

~~$\pi \mid \times \mid \sigma$~~

$$\pi_L(R \times S) \equiv \left(\pi_{A_1 \dots A_n}(R) \right) \times \left(\pi_{B_1 \dots B_m}(S) \right)$$

⑧

$\cup, \cap, -$ ← non-commutative

$$R \cup S \neq S \cup R$$

$$R \cap S \neq S \cap R$$

⑨ Associativity of join, Cartesian product, union and intersection

$$((R \theta S) \theta T) \equiv R \theta (S \theta T)$$

⑩ Commutativity of Projection with Union

$$\pi_L(R \cup S) \equiv \pi_L(R) \cup \pi_L(S)$$

⑪

Transaction Processing - (Next class) 24. 4. 23 (Mon)

Transaction is a logical unit of database. It is a set of tasks to ~~achieve~~ achieve an objective.

Types of Transaction Processing

① Single User and Multi-User

- With single user there is no requirement for concurrent transaction.
- Concurrent transaction takes place in multi-user DBS.

Operations in Transaction

- Insertion
- Deletion
- Modification

- # Each transaction should have a begin and ending.
- # for read operation it is read-only transaction.
- # for write operation it is ~~write-only~~ write-only transaction.

read-item(x) $\rightarrow$ read x into and store in variable x

write-item(x) $\rightarrow$ write value stored in new x into old x

Read-item(x)

- ① Find address of read x
- ② store in buffer
- ③ read from there

Write-item(x)

- ① Find address of disk where x is present
- ② Rewrite the old value of x to the new value of x

(a) Why concurrency control is required?

Ans → Suppose there are 2 transactions

T_1

Problems of Concurrent Transaction

- (1) The lost update problem.
In airline reservation system.
If $x = 80$ and another reservation of 5 seats are done

present value of $x = 75 \leftarrow (80 - 5)$

At the same time 4 seats are cancelled

present value of $x = 75 + 4 = 79$

But

Since both transactions were started simultaneously both transactions read x as 80 and final value of x becomes 84.

* The Updated value of

- (2) The Temporary Update Problem or Dirty Read Problem

③ Incorrect Summary Problem

④ The Unrepeatable Read Problem

⑨ Why Recovery is needed?

Ans

If transaction is finished successfully it is committed

If transaction is unfinished it is aborted.

Types of failure.

- ① Computer failure (System Crash)
- ② transaction error (err i/o)
- ③ Local error or exception conditions detected by the transaction. (Sufficient money in bank but cannot withdraw)
- ④ Concurrency control enforcement
- ⑤

Transaction and System Concepts

Transaction Status and Additional Operations.

* Read From NAVATHE *

Concurrency Control

1.5.31 - (Mon)

Complete Schedule

Schedule $\rightarrow$ History of n transactions (in order)
Where operations of transactions are same same.

For a schedule to be complete,

- (i) Operations in the schedule S are exactly the same transaction t_1 to t_n including commit or abort. ~~of transaction~~
- (ii) For any pair of operations from the same transaction the order of appearance in S is same as the order of appearance in ~~trans.~~ real.
- (iii) For any two conflicting operations one of the two must occur before the other transaction in the schedule.

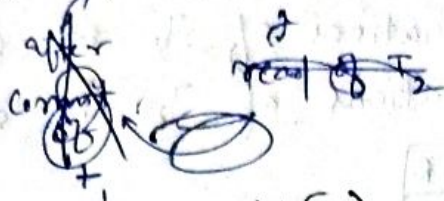
If these 3 conditions are satisfied then schedule is complete.

(iii) $\text{read}(x)_1, \text{write}(x)_2$

Recoverability

A schedule S is recoverable if no transaction t in S commits until all transactions t' that have written an item that t reads have

Q Sa :- $r_1(x), r_2(x); w_1(x), r_1(y); w_2(x), w_1(y)$



Sa :- $r_1(x), r_2(x); c, r_2(x)$ } recoverable schedule

Cascading Rollback or abort

$r_1(x), r_2(x), a_1, w_1(x), r_2(y), r_3(x), r_4(x)$

failure in T_1

all y transaction have to roll back and restart.

$r(x), w_1(x), c_1, r_2(x), w_2(x), c_2, r_3(x)$

Cascadeless,
no need to roll back.

Strict Schedule

Schedule where transaction can neither read nor write a data item x until the last transaction that wrote x is committed or aborted.

$r_1(x), w_1(x), a, r_1(x)$

if $x=10$

$x=10$

$r_1(x), w_1(x), c, r_1(x)$

if $x=10$

$x=x+5$

$x=20$

Initial $x=9$

Q

$S = w_1(x, 5), w_2(x, 8), a_1$

replace with 5

replace with 8

T_1 aborts which means T_2 also aborts

Final value of $x = 9$ before image value

Characterizing Schedules based on Serializability

(i) Execute all the operations of T_1 in seqⁿ followed by all operations of T_2 in seqⁿ

OR

(ii) ~~Execute~~ Execute all the operations of T_1 in sequence.

If we do the schedule in this manner then it is called a serial schedule.

$r_1(x)$

$w_1(x)$

$r_2(x)$

$w_2(x)$

$r_1(y)$

$w_1(y)$

$r_2(y)$

$w_2(y)$

No overlapping transaction.

T_1, T_2, T_1, T_2

OR

T_2, T_1, T_2, T_1

All operations are executed consecutively w/o any overlapping.

Serializable

A schedule with n transactions is serializable if it is equivalent to a ~~serial~~ serial schedule.

Result Equivalant

If two schedules ~~are~~ produce the same result then they are result equivalent.

$$x = 100$$

S1

$$x = i(x)$$

$$x = x + 10$$

$$w = i(x)$$

S2

$$x = i(x)$$

$$x = x + 1$$

$$w = i(x)$$

Conflict Equivalant

For if the order of any two conflicting operation is same in both the schedules.

$$\left. \begin{array}{l} S_1: x_1(x), w_2(x) \\ S_2: x_1(x), w_2(x) \end{array} \right\} \text{Conflict equivalent}$$

Conflict Serializable

A schedule α is said to be conflict serializable if they are conflict equivalent to each other.

Testing Conflict Serializability

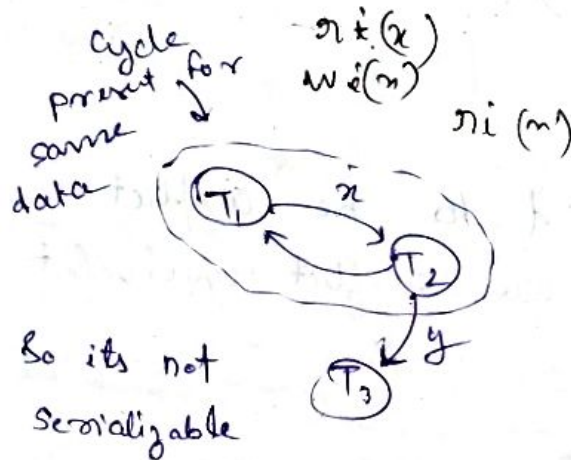
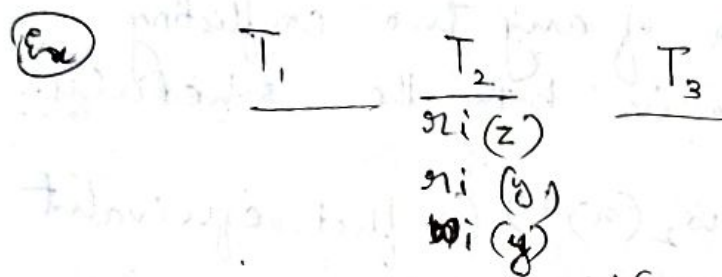
Purpose is, to whether

Precedence / Serializability Graph

① To a node for each transaction T_i draw a node.

② For each case in S where T_i executes a $x.i(x)$ after T_j executes $w.i(x)$, draw an edge T_i to T_j .

- ③ For each case in α where T_j executes $w_i(x)$ after T_i executes $r_i(x)$ then draw edge from T_i to T_j .
- ④ for each case in α where T_j executes $r_i(x)$ after T_i executes $w_i(x)$ then draw edge from T_i to T_j .
- ⑤ Schedule is serializable if there is no cycle in the graph.



View equivalent & view serializable 5.5.23 - Fri

Two schedules s and s' are said to be view equivalent if the following 3 conditions hold:

- ① Same set of transactions participate in s and s' and s and s' include the same operations of those transactions.

- (ii) $r_i(x)$ of T_i in s . If the value of x read by the operation is written by $w_j(x)$ of T_j then the same condition hold good for s' .
- (iii) If the operation $w_k(y)$ of T_k is the last operation to be written in Y in s ~~then~~ then the same is to be for s' .

Constrained Write assumption

- Any write operation $w_i(x)$ in T_i is preceded by $r_i(x)$ in T_i (Value of $w_i(x)$ depends on $r_i(x)$). If that is the case, then it is constrained write assumption.
- If write $w_i(x)$ in T_i is independent then it is blind write.

$T_1 : r_1(x); w_1(x)$ blind constrained

$T_2 : w_2(x)$

$T_3 : w_3(x)$

} Blind Write

Concurrency Control Techniques

(1) Two-Phase - Locking (2PL)

LOCK

Lock is a variable associated with each data item that resembles the state of that particular data item.

~~Types of Lock~~

Types of Locks

① Binary Lock

↳ Locked ($x=1$) cannot be accessed.

↳ Unlocked ($x=0$) can be accessed

Code:- Lock-item(x)

if Lock(x) = 0

then Lock(x) = 1

else

begin

wait (untl Lock(x)) = 0

lock manager wakes up of transaction

Unlock-item(x) → wake up one of transaction
Lock(x) = 0 and assign x

If each ^{lock} item has 2 fields,

< Data-item-name, Lock, Locking-transaction >

Steps for assigning

① If transaction T issues ~~lock~~ Lock-item(x) before $ri(x)$ or $wi(x)$

② After the work is over, the item is to be unlocked. T must issue unlock-item(x) after all read and write operation ~~complete~~ completed.

③ If transaction is already having a lock x

④ Without having the Lock(x), a transaction cannot issue unlock(x)

Shared and Exclusive lock

Main disadvantage of binary lock is that it is either available or not available. To solve this shared and exclusive lock is introduced.

for reading, shared lock is issued and everyone can just read
for writing, exclusive lock is issued [Can't make changes while this lock is activated]

$\langle \text{Data-item name, Lock, No. of reads, Locking} \rangle$
For Trans.

If $\text{Lock}(x) = \text{write-locked}$

then there should be an exclusive lock for x

If $\text{lock}(x) = \text{read-locked}$

then everyone can read but not write.

Rules

① The transaction T must issue the operation for ~~write~~ ^{read} lock(x) before any read item x is performed in $T_i(x)$

② T must issue write lock(x) before any $T_i(x)$ is performed.

③ T must issue unlock(x) after all read and write work over with x .

④ T mustn't issue a read lock(x) if it is already having a lock on $T_i(x)$ or $T_j(x)$

⑤ T must not issue a write lock (x) if it is already holding read lock (x) and write lock (x).

⑥ T will not issue unlock (x) until and unless it is not having a read or write lock (x).

Conversion of Locks $\Rightarrow$ Provision is present in locking

Two-Phase - Locking (2PL)

If all locking operation precede or write the 1st unlock operation then the transaction is 2PL.

<u>T₁</u>	<u>T₂</u>
rl(y)	rl(x)
ri(x)	ri(x)
wl(x)	wl(y)
x = y(y)	un(x)
ri(x)	ri(y)
x = x + y	y = x + 1
wi(x)	
unlock(x)	unlock(y)

* Principle of 2PL

all lock items should be locked before 1st unlock.

Expanding phase
go on adding transaction

Shrinking phase
go on deleting the transaction

Conservative ~~2PL~~ 2PL (static 2PL)

static or conservative 2PL states that all the

Read-set : x, z

write-set : y

All items are to be locked then only the transaction can start.

Advantage

No deadlocks are encountered in (static 2PL)

Deadlocks T_1 is requesting x held by T_2 .
 T_2 is requesting y held by T_1 .

Strict 2PL

The transaction cannot release the write lock until and unless either it is with a successful completion or aborts.

Rigorous 2PL

It cannot release the write and read lock until and unless transaction successful or aborted.

Deadlocks

Dealing with Deadlock ~~or~~ starvation

Starvation $\rightarrow T_1$ requesting for x (x is unavailable) T_2 comes and request for x .
 T_3 and T_4 also requests ~~for~~ for x .

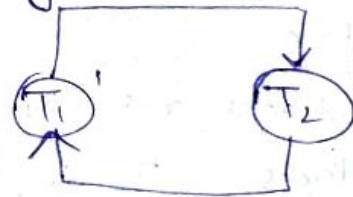
Suppose lock manager by mistakenly assigns x to T_1 . This causes starvation.
Not available to anyone.

Two avoid deadlocks, two protocols are used.

Every transaction locks all items in advance. If it cannot get any item then it must stop. Either all are to be locked or none are to be locked.

~~# There is to be ordering~~

T_1	T_2
rl(y)	
ri(y)	
wl(x)	rl(x)
	ri(x)
	wl(y)



Cycle so two transactions are in deadlock situation.

Schemes to avoid deadlock situation

① wait die scheme

each transaction is to be assigned with a time stamp.

(Ex) T_1 & T_2
 $TS(T_1) < TS(T_2)$
 ↓
 Timestamp

~~Wait-die condition~~

T_i is requesting for a lock in x

and T_j is with T_i causing a conflicting lock.

If $TS(T_i) < TS(T_j)$ — younger
older
with wait die condition

T_i is allowed to wait otherwise the transaction is to be aborted.

If $TS(T_i) > TS(T_j)$
 T_i has to die and restart after some time.

* Wound-Wait

If $TS(T_i) < TS(T_j)$

then T_i wounds T_j and restarts T_j

* No Time Stamp

Also called No-waiting algorithm.

* A no. of trans will abort and restart continuously.

* Cautious Waiting

If T_i tries to lock an item x because but not able to get lock. x is with T_j with a conflicting lock. Then T_j has to wait until T_i has released the lock.

Wait-for graph

To check whether there is a deadlock or not.

To avoid transaction starvation First come first serve is used.